

Variable neighborhood search for the stochastic and dynamic vehicle routing problem

Briseida Sarasola¹ · Karl F. Doerner¹ ·
Verena Schmid¹ · Enrique Alba²

Published online: 14 August 2015
© Springer Science+Business Media New York 2015

Abstract In this paper, the authors consider the vehicle routing problem (VRP) with stochastic demand and/or dynamic requests. The classical VRP consists of determining a set of routes starting and ending at a depot that provide service to a set of customers. Stochastic demands are only revealed when the vehicle arrives at the customer location; dynamic requests mean that new orders from previously unknown customers can be received and scheduled over time. The variable neighborhood search algorithm (VNS) proposed in this study can be extended by sampling for stochastic scenarios and adapted for the dynamic setting. We use standard sets of benchmark instances to evaluate our algorithms. When applying sampling based VNS, on average we were able to improve results obtained by a classical VNS by 4.39 %. Individual instances could be improved by up to 8.12 %. In addition, the proposed VNS framework matches 32 out of 40 best known solutions and provides one new best solution. In the dynamic case, VNS improves on existing results and provides new best solutions for 7 out of 21 instances. Finally, this study offers results for stochastic and dynamic scenarios. Our experiments show that the sampling based dynamic VNS provides better results when the demand deviation is small, and reduces the excess route duration by 45–90 %.

Keywords Variable neighborhood search · Vehicle routing problem · Dynamic requests · Stochastic demands

✉ Briseida Sarasola
briseida.sarasola@univie.ac.at

Karl F. Doerner
karl.doerner@univie.ac.at

Verena Schmid
verena.schmid@univie.ac.at

Enrique Alba
eat@lcc.uma.es

¹ Christian Doppler Laboratory for Efficient Intermodal Transport Operations, Department of Business Administration, University of Vienna, Vienna, Austria

² Departamento de Lenguajes y Ciencias de la Computación, University of Malaga, Málaga, Spain

1 Introduction

Traditionally, routing problems are modeled by assuming that all problem information is known with certainty when planning the routes, and hence is not supposed to change during the execution of the plan. However, many real-world routing applications need to deal with stochastic and/or dynamic information (Psaraftis 1988; Pillac et al. 2013). For example, courier services represent a typical dynamic vehicle routing application that involves a fleet of vehicles collecting packages from customer locations. Service requests can be either arranged in advance or received and planned in real time. This scenario also can be extended to deal with stochastic information about the packages, such as volume or weight, which may not be precisely known in advance. This real-world problem characteristic has motivated the emergence of problem models that incorporate stochasticity and dynamism.

In this paper we consider stochastic customer demands and dynamic requests arising in the vehicle routing problem (VRP) (Dantzig and Ramser 1959). The VRP is an important problem in the field of distribution logistics. It consists of assigning routes at minimal cost to a fleet of vehicles that provide service to a set of geographically distributed customers. This problem is of importance in both academic and industrial domains; on the one hand, the VRP constitutes a generalization of the traveling salesperson problem and is therefore NP-hard (Garey and Johnson 1979); on the other hand, modern companies can benefit from using optimization techniques to reduce costs and provide a better service to their customers.

When dealing with stochastic data, the exact values are not available at the planning stage, although a probability distribution is supposed to be known. The actual value will only be revealed at a later point in time, for example, upon arriving at a customer site. Deterministic data, on the contrary, are known with certainty at the planning stage. Dynamism means that the available information may be revealed or updated over time, as opposed to static information, which does not change. Hence, dynamic problems require replanning, i.e. the routes need to be adjusted or changed, so that dynamic information can be incorporated into the plan. According to the data quality (deterministic or stochastic) and the data evolution (static or dynamic) (Psaraftis 1995) distinctions, we can identify four types of vehicle routing problems (Pillac et al. 2013): deterministic and static, stochastic and static, deterministic and dynamic, and stochastic and dynamic. In this paper we focus on the three last problem types.

Several stochastic VRPs exist, depending on which part of the input is uncertain, such as VRP with stochastic demands (VRPSD) (Secomandi 2001; Mendoza et al. 2010; Bianchi et al. 2006; Erera et al. 2010), VRP with stochastic travel times (Lecluyse et al. 2009; Li et al. 2010; Zhang et al. 2012), and VRP with stochastic service times (Li et al. 2010). There are also several dynamic VRPs according to the source of dynamism. In the VRP with dynamic requests (VRPDR), also simply called dynamic VRP (DVRP), customers can place their orders at any time during the working day, so that their locations and orders become known only after the vehicles have already left the depot (Branchini 2009; Hanshar and Ombuki-Berman 2007; Montemanni et al. 2005). In addition, the travel time for some routes can be modeled using a dynamic component (in addition to its inherent stochasticity) (Potvin et al. 2006; Lorini et al. 2011), and vehicle breakdowns and other problems might also occur (Mu et al. 2011). Combined stochastic and dynamic VRPs have not received a lot of attention in the literature so far. A DVRP with stochastic customers and time windows (VRPTW) has been solved (Bent and Hentenryck 2004), and a real world case study suggests the need to extend the VRPTW to deal with dynamic and stochastic customer information (Hvattum et al. 2006). More recently, and in the field of approximate dynamic programming (ADP), methods to dynamically modify the routing plan have been proposed for the VRPSD with

route duration constraints (Goodson et al. 2013). Also in the field of ADP, a DVRP that aims at maximizing the number of accepted customers was solved by modeling customer locations as random variables (Ulmer et al. 2013).

In this context, investigating vehicle routing problems that deal with both stochastic and dynamic information combines two important real-world application aspects. Thus our main objective in this article is to study a VRP with stochastic demand, dynamic requests, and a combination of both types of customer-related information. We start our analysis with the VRPSD (Laporte et al. 2002; Bianchi et al. 2006), before we tackle the VRPDR (Montemanni et al. 2005; Hanshar and Ombuki-Berman 2007; Khouadjia et al. 2012). Finally, we study the VRP with stochastic demand and dynamic requests (VRPSDDR), which combines features of the two former problems.

When solving dynamic problems, exact methods can provide an optimal solution for only the current (static) problem situation. In this case, the solution is not necessarily optimal after a change occurs, or might not even be a solution to the new problem at hand. Moreover, exact methods, for practical reasons, can deal only with instances of small size. In this context, metaheuristic algorithms are suitable methods that promise good quality solutions in a reasonable amount of time. Regarding the VRP, variable neighborhood search (VNS) is one of the most successful metaheuristic algorithms for solving several types of VRPs (Polacek et al. 2004; Kytöjoki et al. 2007; Kuo and Wang 2012), including DVRP (Sarasola et al. 2011; Khouadjia et al. 2012). Several proposed methods aim to deal with stochastic data in the VRP, including a tabu search (Gendreau et al. 1996), an integer L-shaped exact algorithm (Laporte et al. 2002), reoptimization approaches (Secomandi and Margot 2009), and an adaptive large neighborhood search (Lei et al. 2011). We use a previously introduced stochastic variant of VNS to solve stochastic combinatorial problems (Gutjahr et al. 2007). This framework has also been used successfully to solve routing problems, such as the dynamic stochastic dial-a-ride problem (Schilde et al. 2011).

The contribution of this paper is fivefold. First, we provide four explicit models, which are extended in a straight forward way, and different ways of treating recourse are discussed. Second, we show that a sampling-based VNS provides better results than a classical VNS for the VRPSD when the demand is stochastic and the deviation is small. Third, we show that VNS is competitive with the state-of-the-art algorithms available in the literature; as a result, we obtain an average gap of 0.30 %, match 32 best known solutions out of 40 instances of the VRPSD, and provide one new best solution in this benchmark set. Fourth, we integrate the VNS in a simulation framework to deal with the VRPDR, provide 7 new best solutions out of a set of 21 standard instances, and improve the overall performance in this set of instances. Fifth, we combine stochastic and dynamic solution approaches to solve the VRPSDDR, and show that the sampling based VNS improves on average the results by 3.79 % and reduces the excess route duration by 45–90 %.

The remainder of this article is organized as follows: The VRPs on which we focus are explained in Sect. 2. Section 3 describes the algorithmic techniques, which we evaluate in Sect. 4 using a standard set of benchmarks. Finally, Sect. 5 reports some conclusions and suggestions for further research work.

2 Problem formulation

This section presents the four types of vehicle routing problems, according to the dynamism and/or stochasticity in the related demand and request information (see Table 1). The static

Table 1 Types of vehicle routing problems according to the quality and evolution of the demands information

	Evolution of data	
	Static	Dynamic
<i>Quality of data</i>		
Deterministic	VRP	VRP with dynamic requests
Stochastic	VRP with stochastic demand	VRP with stochastic demand and dynamic requests

and deterministic VRP is formulated in 2.1. This problem is the basis for the static and stochastic VRP, and for the dynamic and deterministic VRP, which are introduced in 2.2 and 2.3, respectively. Finally, the stochastic and dynamic VRP, which is based on the two formerly mentioned problems, is presented in 2.4. Unless stated otherwise, each problem described in 2.2, 2.3, and 2.4, builds upon the model of the related problem(s) introduced before.

2.1 Vehicle routing problem

Consider n customers. Let then $C = \{1, 2, \dots, n\}$ be the set of customers and let the depot be 0, so that the set of all locations is $N = C \cup \{0\}$. D is a matrix of non-negative distances or travel times d_{ij} between all locations i and j in N , satisfying the triangle inequality. Each customer $i \in C$ has a demand q_i and a service time δ_i . A set V of homogeneous vehicles, each with capacity Q , is available at the depot. The fleet size is considered to be large enough to serve all customers.

In addition to standard constraints related to the VRP, we also consider an upper bound on the duration of any route (L), as well as a return time to the depot (T). Constraints like this tend to be more typical in a dynamic and/or stochastic setting. However, for improved readability, we decided to provide these constraints for the static and deterministic case as well. This allows for more consistency between the models to be presented in the next sections. Then, for each vehicle, the total route duration between the departure and return times, including service times, cannot exceed L . Moreover, vehicles can leave the depot at any time, but they should be back at the depot before the closing time T . Late returns are modeled as a soft constraint and penalized in the objective function. The goal is thus to minimize the total cost, which includes both the travel cost and the lateness at the depot.

Decision variables x_{ij}^k are equal to 1 if the edge (i, j) is traversed by vehicle k and 0 otherwise. The amount of time by which vehicle k was delayed with respect to T is λ^k . Variables l_i^k model the time at which vehicle k leaves location i .

The VRP consists of determining a set of vehicle routes of minimal total cost (1), where late returns are modeled as a soft constraint and therefore penalized in the objective function, where α is a lateness penalty coefficient. The routes start and end at the depot, such that every customer in C is visited exactly once by one vehicle (2)–(5). Constraints (6) ensure that a vehicle k spends the right amount of time driving from location i to customer j and serving j , where M can be defined to be a sufficiently large constant. Constraints (7) and (8) guarantee that the total demand of all customers in a route and the total route duration are not exceeded. In addition, vehicles should be back at the depot before the closing time T as ensured by constraints (9). Finally, constraints (10) and (11) impose domain conditions on the variables.

$$\text{minimize} \quad \sum_{i \in N} \sum_{j \in N} \sum_{k \in V} d_{ij} \cdot x_{ij}^k + \alpha \sum_{k \in V} \lambda^k \quad (1)$$

$$\sum_{i \in N} x_{ij}^k - \sum_{i \in N} x_{ji}^k = 0 \quad \forall k \in V, \forall j \in C \quad (2)$$

$$\sum_{i \in N} \sum_{k \in V} x_{ij}^k = 1 \quad \forall j \in C \quad (3)$$

$$\sum_{i \in N} x_{i0}^k = 1 \quad \forall k \in V \quad (4)$$

$$\sum_{j \in N} x_{0j}^k = 1 \quad \forall k \in V \quad (5)$$

$$l_i^k + d_{ij} + \delta_j \leq l_j^k + M(1 - x_{ij}^k) \quad \forall i \in N, \forall j \in C, \forall k \in V \quad (6)$$

$$\sum_{i \in C} \sum_{j \in N} q_i \cdot x_{ij}^k \leq Q \quad \forall k \in V \quad (7)$$

$$\sum_{i \in N} \sum_{j \in N} d_{ij} \cdot x_{ij}^k + \sum_{i \in C} \sum_{j \in N} \delta_i \cdot x_{ij}^k \leq L \quad \forall k \in V \quad (8)$$

$$l_i^k + d_{i0} \leq T + M(1 - x_{i0}^k) + \lambda^k \quad \forall i \in C, \forall k \in V \quad (9)$$

$$x_{ij}^k \in \{0, 1\} \quad \forall i, j \in N, \forall k \in V \quad (10)$$

$$l_i^k \geq 0 \quad \forall i \in N, \forall k \in V \quad (11)$$

2.2 Vehicle routing problem with stochastic demand

The VRPSD is a modification of the VRP introduced in Sect. 2.1. It considers each demand as an independent random variable ξ_i following a known distribution. We assume that all realizations of ξ_i are greater than 0 and they do not exceed the vehicle capacity Q .

The actual demand of each customer is only known once the vehicle arrives at the customer location. Thus, when executing the planned route, route failures can happen, i.e. the planned route may not be feasible any more. The first type of route failure happens when a vehicle is running out of capacity and hence is not able to fully service the current customer i . In this case, the recourse action consists of sending the vehicle back to the depot and notifying the depot to send another vehicle, which continues to serve customer i , and resumes the route from customer i onward as originally planned. We consider two recourse policies to handle serving customer i , depending on whether it is possible to split customer i 's demand or not. Let ϕ_1 be the recourse function for this first type of failure, and specifically, let ϕ_1^{split} be the recourse function when demands are splittable, and ϕ_1^{full} when they are not. Although the original plan might be feasible with respect to T , a second type of route failure could arise after a route failure of the first type, as the backup vehicle might not be able to be back at the depot by T . However, the resulting routes will always be feasible with respect to L , as the triangle inequality is assumed to hold. As before, let ϕ_2 , ϕ_2^{split} and ϕ_2^{full} be the functions that calculate the recourse cost for late arrivals at the depot, where ϕ_2 is the generic recourse function, and *split* and *full* denote whether the demands can be split or not, respectively.

Figure 1 shows an example of the recourse actions for one route, assuming customer demand is splittable. Let node 0 be the depot and nodes one to four be the customers. Let $Q = 4$, $T = L = 5$, $\delta_i = 0$, and $P(\xi_i = 1) = P(\xi_i = 4) = 0.5$, $i \in C$. The original

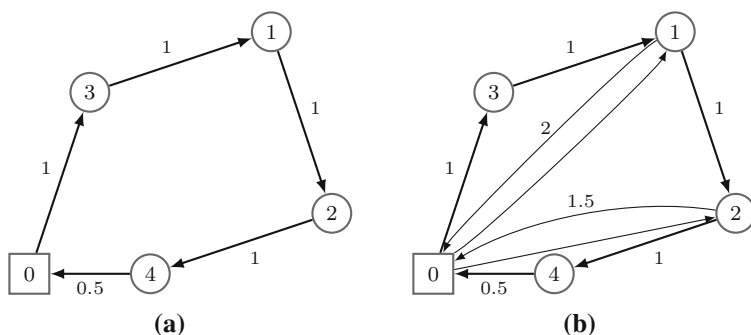


Fig. 1 Example of recourse actions for a single route, where $k = 1$, $Q = 4$, $T = L = 5$, $\delta_i = 0$, and $P(\xi_i = 1) = P(\xi_i = 4) = 0.5$, $i \in C$. The demand realization is 1 for customers three and four and 4 for customers one and two. **a** Original plan. **b** Actual executed plan

plan (left) consists of a single vehicle $k = 1$ leaving the depot at time $t = 0$ and returning at time 4.5 after serving all the customers. The planned route is thus $(0, v_1^1, v_2^1, v_3^1, v_4^1, 0) = (0, 3, 1, 2, 4, 0)$. Let the realization of the customer demands be 1 for customers three and four, and 4 for customers one and two. In such case, the planned route actually fails at two customer locations (customers one and two, v_2^1 and v_3^1 in the route, respectively). The actual plan (right) is executed as follows. The first failure is detected at $t = 2$. Thus, the first vehicle returns to the depot at $t = 4$ and the second vehicle leaves the depot at $t = 2$. The second vehicle finishes serving customer one and continues to customer two. At customer two the second failure occurs at $t = 3$. Then, the second vehicle returns at $t = 4.5$ and the third vehicle leaves the depot at $t = 3$. This last vehicle finishes the service at customer two and continues to serve customer four before returning to the depot at time 6. As a result, the route has a delay of 1 time unit with respect to T .

We model the VRPSD as a multi-stage stochastic programming problem with recourse, where the number of stages is the number of customers in the problem. The formulation presented in Sect. 2.1 is extended as follows. The objective function minimizes the mathematical expectation of the solution cost (12). Constraints (13)–(17) and (19)–(20) are similar to (2)–(6) and (8)–(9), respectively. The expected demand in a route should not exceed the maximum capacity Q (18). Constraints (21) and (22) impose domain conditions on the variables.

$$\text{minimize} \quad \sum_{i \in N} \sum_{j \in N} \sum_{k \in V} d_{ij} \cdot x_{ij}^k + E[\phi_1(x, s, \xi)] + E[\phi_2(x, s, \xi)] \quad (12)$$

$$\sum_{i \in N} x_{ij}^k - \sum_{i \in N} x_{ji}^k = 0 \quad \forall k \in V, \forall j \in C \quad (13)$$

$$\sum_{i \in N} \sum_{k \in V} x_{ij}^k = 1 \quad \forall j \in C \quad (14)$$

$$\sum_{i \in N} x_{i0}^k = 1 \quad \forall k \in V \quad (15)$$

$$\sum_{j \in N} x_{0j}^k = 1 \quad \forall k \in V \quad (16)$$

$$l_i^k + d_{ij} + \delta_j \leq l_j^k + M(1 - x_{ij}^k) \quad \forall j \in C, \forall i \in N, \forall k \in V \quad (17)$$

$$\sum_{i \in C} \sum_{j \in N} E(\xi_i) \cdot x_{ij}^k \leq Q \quad \forall k \in V \quad (18)$$

$$\sum_{i \in N} \sum_{j \in N} d_{ij} \cdot x_{ij}^k + \sum_{i \in C} \sum_{j \in N} \delta_i \cdot x_{ij}^k \leq L \quad \forall k \in V \quad (19)$$

$$l_i^k + d_{i0} \leq T + M(1 - x_{i0}^k) + \lambda^k \quad \forall i \in C, \forall k \in V \quad (20)$$

$$x_{ij}^k \in \{0, 1\} \quad \forall i, j \in N, \forall k \in V \quad (21)$$

$$l_i^k \geq 0 \quad \forall i \in N, \forall k \in V \quad (22)$$

For the first type of recourse action, the expected recourse cost can be calculated as the sum of the recourse costs for each route r_k in the solution, where r_k serves n_r^k customers and is defined as a sequence of vertices $(0, v_1^k, v_2^k, \dots, v_{n_r^k}^k, 0)$. If the recourse action allows customer demand to be split, the expected recourse cost can be calculated as follows:

$$E[\phi_1^{split}(x, s, \xi)] = \sum_{k \in V} \sum_{i=2}^{n_r^k} \sum_{f=1}^{i-1} P\left(\sum_{j=1}^{i-1} \xi_{v_j^k} \leq fQ < \sum_{j=1}^i \xi_{v_j^k}\right) \cdot (d_{v_i 0} + d_{0v_i}) \quad (23)$$

We calculate the expected failure cost like the traditional detour-to-depot recourse action that allows customer demand splitting (Laporte et al. 2002; Christiansen and Lysgaard 2007; Mendoza and Villegas 2013). Please note that splittable demands in the VRPSD refer only to partial deliveries when it is not possible to fully serve a customer due to a route failure. Therefore, partially serving a customer is never planned in the a priori route, contrary to the VRP with split deliveries.

If the recourse action does not allow customer demand to be split, the probability of failing for the $(f+1)$ -th time in the route at a given customer location depends on the location where the f -th failure happened. Related work on the calculation of failure probabilities for non-splittable demands was presented for a recourse strategy involving two vehicles in the VRPSD (Ak and Erera 2007) and for three recourse strategies when solving the adversarial problem (i.e. the one that determines the demand realization that maximizes the actual duration of the a priori route) in the VRPSD with duration constraints (Erera et al. 2010).

As before, let route r_k be a sequence of vertices $(0, v_1^k, v_2^k, \dots, v_{n_r^k}^k, 0)$ with n_r^k customers. The maximum number of failures in the route cannot exceed $n_r^k - 1$, because it is not possible to fail at the first customer v_1 . Let S^{kf} be the set of size $C(n_r^k - 1, f)$ containing every possible way $\pi^{kf} = (\pi_1^{kf}, \pi_2^{kf}, \dots, \pi_f^{kf})$ in which the route r_k could fail f times, where π_i^{kf} is the position in r_k where vehicle k fails for the i -th time. For example, the failure sequence corresponding to Fig. 1 is $\pi^{12} = (2, 3)$. We can thus calculate the expected recourse cost as follows:

$$E[\phi_1^{full}(x, s, \xi)] = \sum_{k \in V} \sum_{f=1}^{n_r^k-1} \sum_{\pi^{kf} \in S^{kf}} P^{full}(\pi^{kf}) \cdot \sum_{\varphi=1}^f \left(d_{v_{\pi_{\varphi}^{kf}}^k 0} + d_{0v_{\pi_{\varphi}^{kf}}^k} \right) \quad (24)$$

where, $P^{full}(\pi^{kf})$, the probability of the failure sequence π^{kf} if customer demand cannot be split, multiplies the cost of back and forth trips to the depot from failure locations. $P^{full}(\pi^{kf})$ can be recursively calculated with help of the F function detailed below, as the product of $F(\pi^{kf}, f)$ multiplied by the probability of being able to serve all customers from position π_f^{kf} to the end of the route with one vehicle. $F(\pi^{kf}, f)$ calculates, beginning at the last failure

position $v_{\pi_f}^{kf}$, the probability of failing at location $v_{\pi_f}^{kf}$ after having failed at the immediately preceding failure location $v_{\pi_{\varphi-1}}^{kf}$. The base case is the probability of the first failure, which is similar to the probability in the ϕ_1^{split} recourse action explained above.

$$P^{full}(\pi^{kf}) = F(\pi^{kf}, f) \cdot P\left(\sum_{j=\pi_f}^{n_r^k} \xi_{v_j^k} \leq Q\right) \quad (25)$$

$$F(\pi^{kf}, \varphi) = \begin{cases} P\left(\sum_{j=1}^{\pi_{\varphi}^{kf}-1} \xi_{v_j^k} \leq Q < \sum_{j=1}^{\pi_{\varphi}^{kf}} \xi_{v_j^k}\right) & \text{if } \varphi = 1 \\ P\left(\sum_{j=\pi_{\varphi-1}^{kf}}^{\pi_{\varphi}^{kf}-1} \xi_{v_j^k} \leq Q < \sum_{j=\pi_{\varphi-1}^{kf}}^{\pi_{\varphi}^{kf}} \xi_{v_j^k}\right) \cdot F(\pi^{kf}, \varphi-1) & \text{if } \varphi > 1 \end{cases} \quad (26)$$

The second cause for infeasibility are late returns of backup vehicles. We model this case by penalizing late returns to the depot, where, for each route, only the last late return is penalized. The depot closing time constraint is similar to the maximum duration of a tour in the VRPSD with duration constraints (Erera et al. 2010). However, note that, because vehicles can leave the depot at any time, and this is important in the dynamic problem, a maximum tour duration $L = T$ does not guarantee to arrive at the depot before its closing time. For example, if $L = T$ and $T > 0$, a vehicle leaving the depot at time $T/2$ and returning at time $3T/2$ does not exceed the maximum tour duration L , but arrives later than T . A related penalty model for the VRPSD with duration constraints was presented in Mendoza et al. (2013).

As before, let route r_k be a sequence of vertices $(0, v_1^k, v_2^k, \dots, v_{n_r^k}^k, 0)$ with n_r^k customers and let S^{kf} be the set of size $C(n_r^k - 1, f)$ containing every possible way $\pi^{kf} = (\pi_1^{kf}, \pi_2^{kf}, \dots, \pi_f^{kf})$ in which the route r_k could fail f times. We can thus calculate the recourse cost as follows:

$$E[\phi_2(x, s, \xi)] = \sum_{k \in V} \sum_{f=1}^{n_r^k-1} \sum_{\pi^{kf} \in S^{kf}} P(\pi^{kf}) \cdot \max(0, \text{return}^{\pi^{kf}} - T) \quad (27)$$

where $P(\pi^{kf})$ is the probability of the failure sequence π^{kf} and $\text{return}^{\pi^{kf}}$ is the return time of vehicle k after failing according to π^{kf} . If demand splitting is not allowed, we calculate $P^{full}(\pi^{kf})$ using (25). Otherwise, $P^{split}(\pi^{kf})$ can be calculated as follows:

$$P^{split}(\pi^{kf}) = \prod_{\varphi=1}^f P\left(\sum_{j=1}^{\pi_{\varphi}^{kf}-1} \xi_{v_j^k} \leq \varphi Q < \sum_{j=1}^{\pi_{\varphi}^{kf}} \xi_{v_j^k}\right) P\left(\sum_{j=1}^{n_r^k} \xi_{v_j^k} \leq (f+1)Q\right) \quad (28)$$

The return time of vehicle k is the arrival time at the depot after the last failure in the route. Let $\tilde{t}_f^{\pi^{kf}}$ be the time at which vehicle k fails for the f -th and last time in route r_k according to the failure sequence π^{kf} , so that, at this time, a new vehicle is sent from the depot to complete the service to the remaining part of the route. The return time at the depot can be calculated as follows:

$$\text{return}^{\pi^{kf}} = \tilde{t}_f^{\pi^{kf}} + d_{0v_f^k} + \delta_{v_f^k} + \sum_{j=f}^{n_r^k-1} (d_{v_j^k v_{j+1}^k} + \delta_{v_{j+1}^k}) + d_{v_{n_r^k}^k 0} \quad (29)$$

Note that the corresponding service time at a given failure location is fully completed by the recourse route. Therefore, we assume that failures are detected as soon as the vehicle arrives at the customer location and before beginning to serve it. The value of $\tilde{t}_f^{\pi^{kf}}$ can be thus recursively calculated as follows:

$$\tilde{t}_\varphi^{\pi^{kf}} = \begin{cases} t_0^k + d_{0v_1^k} + \delta_{v_1^k} + \sum_{j=1}^{\pi_1^{kf}-1} (d_{v_j^k v_{j+1}^k} + \delta_{v_{j+1}^k}) - \delta_{v_{\pi_1^{kf}}^k} & \text{if } \varphi = 1 \\ \tilde{t}_{\varphi-1}^{\pi^{kf}} + d_{0v_{\pi_{\varphi-1}^{kf}}^k} + \delta_{v_{\pi_{\varphi-1}^{kf}}^k} + \sum_{j=\pi_{\varphi-1}^{kf}}^{\pi_\varphi^{kf}-1} (d_{v_j^k v_{j+1}^k} + \delta_{v_{j+1}^k}) - \delta_{v_{\pi_\varphi^{kf}}^k} & \text{if } \varphi > 1 \end{cases} \quad (30)$$

2.3 Vehicle routing problem with dynamic requests

The particularity of the VRPDR arises because customers appear dynamically during a work day. Assuming the depot is open during the work day $[0, T]$, each customer i has an associated available time $\tau_i \in [0, T]$, which indicates when both the customer location and demand become known: static requests are such that $\tau_i = 0$, whereas for dynamic requests, $\tau_i \in (0, T]$. The dynamic requests need to be scheduled, either by inserting them into existing routes or by creating new ones. The objective is to minimize the total cost, which includes both the travel cost and the lateness at the depot.

We follow a rolling-horizon procedure by splitting the planning horizon of length T in a sequence of m equidistant time intervals, each of length p , where $p = T/m$. We model the dynamic problem P as a sequence of static sub-problems $(P_0, P_1, \dots, P_{m-1})$, such that each P_h , $0 \leq h < m$, aims at optimizing the problem using the information known at time $t = h \cdot p$.

We assume that P_0 is the classical CVRP defined in Sect. 2.1. Sub-problem P_h , $h > 0$, is defined as follows. Let C be the set of customers known in P_{h-1} , such that $\tau_i \in [0, t - p)$, $i \in C$, and 0 is the depot. Assuming that n^+ new customers, such that $\tau_i \in [t - p, t)$ for all new customer i , are revealed, let $C^+ = \{n + 1, \dots, n + n^+\}$ be the set of new customers to be considered for the optimization at time t . Note that, if it can be guaranteed that $t + d_{0i} + \delta_i + d_{i0} \leq T$ for all customers $i \in C^+$ then it would be feasible to serve all new customers without violating T . Let C' be the set of all customers $C' = C \cup C^+$ and N' the set of all locations $N' = C' \cup \{0\}$. Let d_{ij} be the distance between i and j , for all $i, j \in N'$. Let q_i and δ_i be the demand and service time of customer $i \in C^+$. In addition, let $\hat{x}_{ij}^k$ be equal to 1 if vehicle k was supposed to travel from location i to j according to the solution of the previous problem P_{h-1} , and let $\hat{t}_i^k$ denote the time at which the vehicle k had to leave location i in P_{h-1} .

P_h , $h > 0$, consists of determining a set of vehicle routes of minimal total cost (31). The routes start and end at the depot, such that every customer in C' is visited exactly once by one vehicle (32)–(35). Constraints (36)–(39) are similar to (6)–(9). Constraints (40) enforce that any decision taken before time t is not changed. Constraints (41) and (42) define the domains of the variables.

$$\text{minimize} \quad \sum_{i \in N'} \sum_{j \in N'} \sum_{k \in V} d_{ij} \cdot x_{ij}^k + \alpha \sum_{k \in V} \lambda^k \quad (31)$$

$$\sum_{i \in N'} x_{ij}^k - \sum_{i \in N'} x_{ji}^k = 0 \quad k \in V, j \in C' \quad (32)$$

$$\sum_{i \in N'} \sum_{k \in V} x_{ij}^k = 1 \quad j \in C' \quad (33)$$

$$\sum_{i \in N'} x_{i0}^k = 1 \quad k \in V \quad (34)$$

$$\sum_{j \in N'} x_{0j}^k = 1 \quad k \in V \quad (35)$$

$$l_i^k + d_{ij} + \delta_j \leq l_j^k + M(1 - x_{ij}^k) \quad j \in C', i \in N', k \in V \quad (36)$$

$$\sum_{i \in C'} \sum_{j \in N'} q_i \cdot x_{ij}^k \leq Q \quad k \in V \quad (37)$$

$$\sum_{i \in N} \sum_{j \in N} d_{ij} \cdot x_{ij}^k + \sum_{i \in C} \sum_{j \in N} \delta_i \cdot x_{ij}^k \leq L \quad \forall k \in V \quad (38)$$

$$l_i^k + d_{i0} \leq T + M(1 - x_{i0}^k) + \lambda^k \quad i \in C', k \in V \quad (39)$$

$$x_{ij}^k = \hat{x}_{ij}^k \quad i, j \in C, k \in V, \text{ where } \hat{l}_i^k < t \wedge \hat{x}_{00}^k = 0 \quad (40)$$

$$x_{ij}^k \in \{0, 1\} \quad \forall i, j \in N', \forall k \in V \quad (41)$$

$$l_i^k \geq 0 \quad \forall i \in N', \forall k \in V \quad (42)$$

2.4 Vehicle routing problem with stochastic demand and dynamic requests

This case represents a combination of the two former problems: Each request becomes known at time $\tau_i \in [0, T]$, and the associated demand is a random variable ξ_i . The objective function is to minimize the expected total cost.

As introduced in Sect. 2.3, we follow a rolling horizon-procedure. The dynamic problem P is considered a sequence of static subproblems $(P_0, P_1, \dots, P_{m-1})$, such that P_0 is the VRPSD defined in Sect. 2.2, and P_h , for all $h > 0$, is the optimization problem that will be solved at time $t = h \cdot p$ and defined as follows.

As before, C is the set of customers known in $[0, t - p)$, and 0 is the depot, C^+ is the set of customers revealed in $(t - p, t]$, C' is the set of all customers $C' = C \cup C^+$, and N' is the set of all locations $N' = C' \cup \{0\}$. Likewise, d_{ij} is the distance between i and j , $i, j \in N'$, ξ_i is the random demand and δ_i is the service time of customer $i \in C^+$. Let C^S be the set of customers that have already been served or are already planned to be served in P_h . Let $\hat{Q}^k$ be the available capacity of vehicle k to serve the remaining part of the route beginning at the next unplanned customer. Let $\hat{x}_{ij}^k$ be equal to 1 if vehicle k was supposed to travel from location i to j in the previous solution, and $\hat{s}_i^k$ contains the time at which the vehicle k is ready to leave location i . Please note that, in case of a failure, $\hat{s}_i^k$ represents the time at which vehicle k has completed service at customer i according to the recourse actions.

P_h , $h > 0$, consists of determining a set of vehicle routes of minimal expected cost (see (12) in Sect. 2.2), where constraints (32)–(36) and (38)–(42) are the same as in Sect. 2.3. Constraints (37) are replaced by (43), which ensure that the available vehicle capacity is sufficient to serve the expected demand in the unserved part of the route.

$$\sum_{i \in C' \setminus C^S} \sum_{j \in N} E(\xi_i) \cdot x_{ij}^k \leq \hat{Q}^k \quad k \in V \quad (43)$$

The expected cost of recourse is calculated as described in Sect. 2.2.

3 Solution techniques

We study ways to adapt VNS to a VRP scenario in which customer demands can be deterministic or stochastic and customer requests can be static or dynamic. First, we designed a fast and competitive VNS algorithm to deal with the standard VRP. Second, we included our VNS in the S-VNS framework to tackle the VRPSD. Third, we adapted the VNS to dynamic environments by embedding it into a scheduler that considers the changes that occur during the work day simulation. Fourth and finally, we present an approach to merge the dynamic and the stochastic frameworks to deal with the VRPSDDR.

3.1 Variable neighborhood search

Variable neighborhood search (VNS) is a well-known metaheuristic (Hansen and Mladenović 1999), based on the principle of systematically changing neighborhoods to escape from local optima. Thus, VNS consists of defining a succession of κ_{max} neighborhood structures $\mathcal{N}_1(s)$, $\mathcal{N}_2(s)$, ..., $\mathcal{N}_{\kappa_{max}}(s)$ around a solution s in the search space. At each iteration, another solution s' gets generated from the current neighborhood $\mathcal{N}_{\kappa}(s)$ and improved through the local search routine, which produces a new local minimum s'' . The latter is accepted at the move-or-not step depending on the acceptance criterion (in the standard VNS, $f(s'') \leq f(s)$), s'' becomes the new current incumbent solution, and the search continues in $\mathcal{N}_1(s'')$; otherwise, the search moves in the next iteration to the next neighborhood $\mathcal{N}_{\kappa+1}(s)$. Additionally, the best found solution can be saved in $\hat{s}$ and is always replaced by s if $f(s) < f(\hat{s})$. Algorithm 1 shows the outline of a standard VNS.

Algorithm 1 Outline of standard VNS

```

1:  $s \leftarrow initialize()$ 
2:  $\hat{s} \leftarrow s$ 
3: while termination condition not met do
4:    $\kappa \leftarrow 1$ 
5:   while  $\kappa < \kappa_{max}$  do
6:      $s' \leftarrow shake(\mathcal{N}_{\kappa}(s))$ 
7:      $s'' \leftarrow localSearch(s')$ 
8:     if  $moveOrNot(s, s'')$  then
9:        $s \leftarrow s''$ 
10:     $\kappa \leftarrow 1$ 
11:    if  $f(s) < f(\hat{s})$  then
12:       $\hat{s} \leftarrow s$ 
13:    end if
14:  else
15:     $\kappa \leftarrow \kappa + 1$ 
16:  end if
17: end while
18: end while

```

To modify the VNS for a particular problem, it is necessary to define the initialization phase, the set of neighborhood structures, and the local search procedure. In addition, we modify the acceptance criterion used in the standard VNS. We build initial solutions using the Clarke and Wright savings algorithm (Clarke and Wright 1964). The shake procedure uses the CROSS (Taillard et al. 1997) and the iCROSS operators (Bräysy 2003) as explained in Kritzing et al. (2012), which consist of exchanging two segments of different routes. The current neighborhood κ defines the maximum length of the segments, which is actu-

ally calculated as $\min(p, n_r^k)$, where $p \sim U(1, \kappa)$ and n_r^k is the number of customers in each selected route. Our local search uses two improving procedures: first, we use a 2-opt heuristic (Croes 1958), which inverts sequences of one single route; second, we use a 2-opt* heuristic (Potvin and Rousseau 1995), which exchanges the ending segments of two different routes. A first improvement strategy is used, which means that the local search restarts immediately when an improvement is found. Each local search runs until no more improvement is possible. The 2-opt local search runs first, its output solution being the input to the 2-opt* local search. Afterwards the algorithm exits the local search procedure (there is no return to the 2-opt method at this point). Only routes modified in the shaking procedure undergo the local search method.

Because accepting only improving solutions can hinder the algorithm from escaping from local optima (Polacek et al. 2004), we also consider the potential acceptance of poorer solutions if certain conditions are met. At the move-or-not step (line 8), an improving solution is always accepted, whereas a non-improving one is accepted after a minimum number of iterations counted from the last accepted move, only if the cost of the non-improving solution does not exceed a threshold θ , defined as a percentage of the incumbent solution cost.

The incumbent solution is repaired before its evaluation, so it complies with the problem constraints. Our repair procedure consists of choosing the best possible split in an infeasible route (i.e., the split that obtains routes with minimal costs). If a split results in a route with a single customer, the algorithm tries to insert it in another existing route using the cheapest insertion operator.

3.2 Stochastic VNS

We embed our VNS in the S-VNS framework proposed by Gutjahr et al. (2007) (pseudocode in Algorithm 2). The S-VNS shares the general structure of the classic VNS; the main difference pertains to the way their solutions are evaluated and compared. On the one hand, the classic VNS compares two solutions using deterministic information about the solution cost, both in the local search (with the possibility of using partial costs instead of reevaluating the whole solution) and in the move-or-not step. On the other hand, the S-VNS makes use of stochastic information to compare solutions by drawing a sample of scenarios of the unserved customer demands. The current solution gets evaluated in each scenario; the average of these objective function values is the sample average estimator (SAE).

The SAE applies whenever a comparison is required (see lines 13, 22, and 29 in Algorithm 2). At the local search step, a set of $size_0$ demand scenarios is used to find the best neighbor $\bar{s}^*$ of the current local search solution $\bar{s}$ and to compare them. The local search runs until it finds a local optimum or a maximum number ρ_{max} of steps is reached. At the move-or-not step, the result of the local search s'' is evaluated against the incumbent solution s using a sample Z' of $size_0$ scenarios. S-VNS incorporates a tournament step to ensure convergence. The tournament takes place at the end of each iteration and consists of drawing a sample of size $size_m$ to compare the current incumbent solution after the move-or-not step s and the global best solution (i.e., best-so-far solution) $\hat{s}$.

We adapted the S-VNS to the VRPSD in the following way: The initializing, shaking, and local search phases remain as described in Sect. 3.1. Although the original S-VNS framework uses the SAE to evaluate neighbors during the local search, we have observed that, in our case, this approach consumes execution time without improving the results. We use the SAE in the move-or-not step and in the tournament phase. In addition, we modify the original S-VNS by setting the sample size $size_m$ to a constant value instead of increasing it according to the number of iterations, as proposed in Gutjahr et al. (2007). We have observed that a

Algorithm 2 Outline of stochastic VNS

```

1:  $s \leftarrow \text{initialize}(\{i \mid i \in C\})$ 
2:  $\hat{s} \leftarrow s$ 
3: while termination condition not met do
4:    $\kappa \leftarrow 1$ 
5:   while  $\kappa < \kappa_{\max}$  do
6:      $s' \leftarrow \text{shake}(N_{\kappa}(s))$ 

7:      $\bar{s} \leftarrow s'$ 
8:      $\rho \leftarrow 1$ 
9:     local-optimum-found  $\leftarrow$  false
10:    repeat
11:       $Z \leftarrow \text{sample}(\text{size}_0)$ 
12:       $\bar{s}^* \leftarrow \text{bestNeighbor}(\bar{s}, Z)$ 
13:      if  $\text{moveOrNotSAE}(\bar{s}, \bar{s}^*, Z)$  then
14:         $\bar{s} \leftarrow \bar{s}^*$ 
15:         $\rho \leftarrow \rho + 1$ 
16:      else
17:        local-optimum-found  $\leftarrow$  true
18:      end if
19:    until local-optimum-found or  $\rho > \rho_{\max}$ 
20:     $s'' \leftarrow \bar{s}$ 

21:     $Z' \leftarrow \text{sample}(\text{size}_0)$ 
22:    if  $\text{moveOrNotSAE}(s, s'', Z')$  then
23:       $s \leftarrow s''$ 
24:       $\kappa \leftarrow 1$ 
25:    else
26:       $\kappa \leftarrow \kappa + 1$ 
27:    end if

28:     $Z'' \leftarrow \text{sample}(\text{size}_m)$ 
29:    if  $\text{moveOrNot}(\hat{s}, s, Z'')$  then
30:       $\hat{s} \leftarrow s$ 
31:    end if
32:  end while
33: end while

```

small value of size_m allows the algorithm to converge faster, allocating more time to the optimization process and less time to compare solutions.

Because the capacity constraint may be violated, a recourse action is required to correct the routing plan. The recourse action consists of sending the vehicle back to the depot when it arrives at a customer location and cannot fulfill the demand. Demand splitting is not allowed, so the vehicle must return to the depot and then go back to serve the remaining unserved subroute. After a recourse action a solution is always feasible, because it never violates either the capacity or the tour duration constraints.

However, calculating recourse actions lead to increased computation times. For this reason, we propose three ways to deal with infeasible solutions. The first is based on our deterministic VNS described in Sect. 3.1: A *correct-all* strategy consists of correcting each solution in each of the size_0 sample scenarios before it is evaluated. This method guarantees that a feasible solution is evaluated each time, but it is very time consuming compared with other possible strategies. The second one is a *penalize-all* strategy that consists of using penalty factors for broken constraints. We use two penalty parameters β and γ to penalize the capacity excess $Q'(s)$ and the exceeded traveling time $T'(s)$:

$$f_{new}(s) = f(s) + \beta Q'(s) + \gamma T'(s) \quad (44)$$

The third method is a combined strategy that we call *correct-one-penalize-all*. With it, we propose to correct each solution once before computing its SAE, using the expected value of the demand distributions. A solution can then be evaluated in each of the $size_0$ samples using penalties, as expressed in Eq. 44. Assuming that the deviation of the demand distributions is not large, a route that is infeasible in a sample scenario ω_1 will most likely be infeasible in another sample scenario ω_2 , because stochastic scenarios are drawn from the same demand distributions and do not differ completely. This method avoids repeating similar reparation steps in each sample, while still taking into account the constraint violations for each scenario.

3.3 Dynamic VNS

To tackle the VRP with dynamic requests, it is necessary to combine the optimization algorithm with an additional component that manages the simulation. Following the architecture proposed by Montemanni et al. (2005), this component is an event scheduler, which receives new requests and manages the simulation time.

One standard approach to deal with dynamic customers is to divide the work day T into m time periods with equal length T/m , such that new requests received in the current time period get processed at the end of the time period (Hanshar and Ombuki-Berman 2007; Montemanni et al. 2005). An alternative strategy is to schedule each new customer request as soon as the request is known (Gendreau et al. 1999), which may be necessary when dealing with urgent requests and/or customers with time window constraints. However, the early insertion of customers does not guarantee a better solution (Montemanni et al. 2005).

Algorithm 3 shows the pseudocode for our D-VNS, solving the VRPDR. Initially, the event scheduler sets the simulation time T_s and the current time period to 0 (lines 1–2), builds an initial solution using static customers, that is, the orders known in advance (line 5), and starts the optimization algorithm (lines 9–26). When the time assigned to the current time period is over, the event scheduler obtains a feasible solution from the optimization method (line 27) and commits vehicles to customers with processing times in the next T/m time units (line 28). The processing time p_i of customer i is the time when the vehicle leaves from the previous location in the route. In the following time periods, this process repeats (lines 29–30 and back to line 3), using the solution from the previous time period to build the initial solution, together with the new customers that appeared in the last time period, which get scheduled into new routes using the Clarke and Wright heuristic (line 7). The optimization algorithm only allows to modify those parts of the routes that are not planned to be served by vehicles in the current time period. After the decision has been made to serve a customer using a specific vehicle in the current time period, this decision cannot be changed.

3.4 Dynamic stochastic VNS

This section discusses some additional required modifications to design a solution for the VRP with stochastic demand and dynamic requests.

The stochastic event scheduler works in a way similar to the deterministic event scheduler described in Sect. 3.3, except for the calculation of real demand when a vehicle must effectively serve a customer. That is, customer demand remains uncertain until a vehicle arrives at the customer location and must serve it. In this step, customer demand is sampled in the customer distribution, according to the expected value and deviation. From this moment, demand is no longer stochastic but rather becomes deterministic and fixed. If possible, the

Algorithm 3 Outline of dynamic VNS

```

1:  $T_s \leftarrow 0$ 
2:  $h \leftarrow 0$ 
3: while  $T_s < T$  do
4:   if  $T_s = 0$  then
5:      $s \leftarrow \text{initialize}(\{i \mid \tau_i = 0\})$ 
6:   else
7:      $s \leftarrow s^{h-1} \cup \text{initialize}(\{i \mid T_s - \frac{T}{m} \leq \tau_i < T_s\})$ 
8:   end if
9:    $s^* \leftarrow s$ 
10:  while termination condition not met do
11:     $\kappa \leftarrow 1$ 
12:    while  $\kappa < \kappa_{\max}$  do
13:       $s' \leftarrow \text{shake}(N_\kappa(s))$ 
14:       $s'' \leftarrow \text{localSearch}(s')$ 
15:       $s'' \leftarrow \text{repair}(s'')$ 
16:      if  $\text{moveOrNot}(s, s'')$  then
17:         $s \leftarrow s''$ 
18:         $\kappa \leftarrow 1$ 
19:        if  $f(s) < f(s^*)$  then
20:           $s^* \leftarrow s$ 
21:        end if
22:      else
23:         $\kappa \leftarrow \kappa + 1$ 
24:      end if
25:    end while
26:  end while

27:  $s^h \leftarrow s^*$ 
28:  $\text{commit}(s^h, \{i \mid p_i \in (T_s, T_s + \frac{T}{m}]\})$ 
29:  $h \leftarrow h + 1$ 
30:  $T_s \leftarrow T_s + \frac{T}{m}$ 
31: end while

```

assigned vehicle serves the customer with calculated demand; otherwise, the vehicle returns to the depot. At this point, a new vehicle gets released from the depot to serve the remainder of the unserved route. Another possible recourse action is to send the current vehicle back to the depot to unload; however, we work with instances that do not restrict the number of available vehicles, and an early vehicle dispatch can help avoid depot time window constraint violations.

We also emphasize that a committed solution should always be feasible. The only exception is when it is not possible to return to the depot within its time window, due to recourse actions at the end of the planning period. We offer a further relevant discussion in Sect. 4.6.

Algorithm 4 shows the pseudocode for our SD-VNS solving the VRP with stochastic demand and dynamic requests.

4 Performance assessment

In this section, we explain the test instances, describe the experimental settings, and present the results obtained for each vehicle routing scenario described in Sect. 2. For that purpose, we describe the instance sets and present some common settings in Sects. 4.1 and 4.2 and then proceed in four steps: (1) we analyze our VNS over a well-known set of CVRP instances in Sect. 4.3, (2) we approach the VRPSD in Sect. 4.4 by evaluating both our VNS and S-

Algorithm 4 Outline of dynamic stochastic VNS

```

1:  $T_s \leftarrow 0$ 
2:  $h \leftarrow 0$ 
3: while  $T_s < T$  do
4:   if  $T_s = 0$  then
5:      $s \leftarrow \text{initialize}(\{i \mid \tau_i = 0\})$ 
6:   else
7:      $s \leftarrow s^{h-1} \cup \text{initialize}(\{i \mid T_s - \frac{T}{m} \leq \tau_i < T_s\})$ 
8:   end if

9:    $\hat{s} \leftarrow s$ 

10:  while termination condition not met do
11:     $\kappa \leftarrow 1$ 
12:    while  $\kappa < \kappa_{\max}$  do
13:       $s' \leftarrow \text{shake}(N_\kappa(s))$ 

14:       $\bar{s} \leftarrow s'$ 
15:       $\rho \leftarrow 1$ 
16:      local-optimum-found  $\leftarrow$  false
17:      repeat
18:         $Z \leftarrow \text{sampleDemands}(\text{size}_0)$ 
19:         $\bar{s}^* \leftarrow \text{bestNeighbor}(\bar{s}, Z)$ 
20:        if  $\text{moveOrNotSAE}(\bar{s}, \bar{s}^*, Z)$  then
21:           $\bar{s} \leftarrow \bar{s}^*$ 
22:           $\rho \leftarrow \rho + 1$ 
23:        else
24:          local-optimum-found  $\leftarrow$  true
25:        end if
26:      until local-optimum-found or  $\rho > \rho_{\max}$ 
27:       $s'' \leftarrow \bar{s}$ 

28:       $Z' \leftarrow \text{sampleDemands}(\text{size}_0)$ 
29:      if  $\text{moveOrNotSAE}(s, s'', Z')$  then
30:         $s \leftarrow s''$ 
31:         $\kappa \leftarrow 1$ 
32:      else
33:         $\kappa \leftarrow \kappa + 1$ 
34:      end if

35:       $Z'' \leftarrow \text{sampleDemands}(\text{size}_m)$ 
36:      if  $\text{moveOrNotSAE}(\hat{s}, s, Z'')$  then
37:         $\hat{s} \leftarrow s$ 
38:         $\kappa \leftarrow 1$ 
39:      else
40:         $\kappa \leftarrow \kappa + 1$ 
41:      end if
42:    end while
43:  end while

44:   $s^h \leftarrow s^*$ 
45:   $\text{commit}(s^h, \{i \mid p_i \in (T_s, T_s + \frac{T}{m}]\})$ 
46:   $h \leftarrow h + 1$ 
47:   $T_s \leftarrow T_s + \frac{T}{m}$ 
48: end while

```

VNS, and comparing S-VNS with the best state-of-the art algorithms for VRPSD, (3) we compare D-VNS with the best state-of-the-art algorithms for VRPDR in Sect. 4.5, and (4) in Sect. 4.6 we test D-VNS and SD-VNS over the VRPSDDR to analyze whether taking stochastic information into account is convenient and useful in terms of solving stochastic and dynamic problem versions.

4.1 Test instances

To evaluate the VNS with the standard capacitated VRP (i.e., all demands are exactly known in advance) and compare it with other algorithms in prior literature, we used the 14 instances defined in the Christofides et al. benchmark (1979), which featured 50–199 customers. Because some of these instances define a maximum tour duration and no vehicle leaves the depot after time 0, we set $T = L$ (depot closing time equal to the maximum tour duration). In this case, constraints (9) in Sect. 2.1 can be dropped, and no lateness penalty factors λ^k are required.

To evaluate the performance of our methods for the stochastic and/or dynamic VRP, we used the set of benchmark instances defined by Kilby (1998) and based on classical instances from Christofides et al. (1979), Fisher and Jaikumar (1981), and Taillard (1993). These instances contain 50–199 customers, and were extended by Kilby with a uniformly random distributed available time for each customer, a service time at the customer location, and a work day duration, chosen to be eight times the maximum distance between pairs of visits.

When we solve the VRPSD using the Kilby instances, we consider that all customers are static and known in advance (their available time is 0). We extend the instances by considering that the demands can be drawn using a uniform distribution with mean μ and standard deviation σ , as in previous work on the VRPSD (Bianchi et al. 2006; Secomandi 2001). To keep the problem similar to the standard VRP detailed above, we treat Kilby's work day duration as L . Therefore, we can drop constraints (20) in Sect. 2.2 and no recourse action ϕ_2 is necessary.

Regarding the VRPDR, we used the set of instances defined by Kilby and set T equal to the work day duration. Therefore, we can drop constraints (38) (maximum route duration) in Sect. 2.3. In addition, we follow a common approach, considering customers with $\tau_i < T \cdot 0.5$ as dynamic, while the rest are static and known at the beginning of the simulation. Due to the instance's construction, it is guaranteed that it is possible to serve the dynamic customers without breaking the depot closing time constraint.

In the VRPSDDR, we used the Kilby benchmark by extending it with uniformly distributed customer demands as explained above for the VRPSD. As in the VRPDR case, we can drop the maximum tour duration constraint. However, in this case we must consider that, due to recourse actions, late arrivals at the depot are possible. Therefore, both ϕ_1 and ϕ_2 are necessary.

Finally, we also evaluate the performance of our VNS framework for the VRPSD using the set of benchmark instances defined by Christiansen and Lysgaard (2007) and based on the Augerat test sets A and P (Augerat et al. 1998) and the Christofides and Eilon test set (1969) with up to 60 customers. Christiansen and Lysgaard extended them with poisson distributed customer demands, where the expected demand is set equal to the original deterministic demand. Because they do not define a maximum tour duration nor a depot closing time, we can drop constraints (19) and (20) in Sect. 2.2, and therefore, no recourse action ϕ_2 is necessary.

4.2 Experimental settings

We set the VNS parameters during a preliminary testing phase. For the deterministic algorithms (both VNS and D-VNS), we determined that using a maximum neighborhood size $\kappa = 5$ yields the most competitive results for this set of instances. This result matches the maximum neighborhood size used in previous work pertaining to the dynamic stochastic dial-a-ride problem (Schilde et al. 2011). In addition, worse solutions may be accepted after 500 iterations since the last acceptance of a solution, if their cost does not exceed the cost of the incumbent solution by more than $\theta = 5\%$. In the stochastic algorithms (S-VNS and SD-VNS), we reuse the same κ and θ , and, in Sects. 4.4.1 and 4.6, set the sample size to $size_0 = 50$ and the tournament sample size to $size_m = 1$. We checked experimentally whether using $size_m = 1$ was better than not using the tournament at all, or better than using bigger tournament sample sizes. In Sect. 4.4.2, and because we are comparing with existing methods in the literature that use an analytical calculation of the expected cost, the S-VNS calculates the exact expected cost using the algorithm explained in Goodson et al. (2012).

Our VNS approaches were implemented in Java 1.5. The experiments in Sects. 4.3, 4.4.1, 4.5, and 4.6 ran on a single core of an Intel Core2 Quad 2.6GHz machine with 4 GB memory and we performed 30 independent runs for each experiment. The experiments in Sect. 4.4.2 ran on a single core of an Intel Core i7 3.4 GHz machine with 8 GB memory and, following previous work on this benchmark instances, we performed 10 independent runs for each experiment.

4.3 Standard capacitated VRP

We compare our VNS algorithm over the Christofides et al. benchmark set with the best solution costs obtained through state-of-the-art methods, as well as the best and average results obtained by a competitive VNS over a broad set of VRP instances, such as the unified VNS (Kritzing et al. 2012). Following previous settings found in the literature (Zachariadis and Kiranoudis 2010; Kritzing et al. 2012), we allow the algorithm to run for a maximum time of 600 CPU seconds. We provide these results in Table 2, along with the best and average gaps with the best known solution and the unified VNS. Our VNS identifies the best known solution in 8 of 14 instances and is able to find it for all 30 runs in 5 of these 8 instances. The best solutions found by VNS are, on average, only 0.24% worse than the best ones available in prior literature, and the average solution cost is close to the best known cost for most instances (the average gap is 0.41%). Compared with the unified VNS, we obtain low gap values (0.17 and 0.13% for the best and the average, respectively). The average solution quality is very similar, with gaps in the range $[-0.28, 0.20\%]$ for all instances except those with 199 customers (instances 5 and 10).

4.4 VRP with stochastic demand

This section aims both to analyze the effectiveness of the sampling mechanism in S-VNS by comparing it with the standard VNS, and to evaluate the performance of our VNS approach on a standard set of benchmarks for the VRPSD.

4.4.1 Analysis of the sampling mechanism

Both VNS and S-VNS are run for 60s on each instance of the Kilby benchmark set. VNS solves the problem using mean demand values, whereas S-VNS samples the expected demand

Table 2 Best and average route length obtained in the static and deterministic VRP (Christofides instances)

I.	VNS		Best-known			Unified VNS			
	Best	Avg	Cost	Best gap	Avg. gap	Best cost	Avg. cost	Best gap	Avg. gap
1	524.61	524.61	524.61	0	0	524.61	524.61	0	0
2	835.26	835.26	835.26	0	0	835.26	835.27	0	0
3	826.94	828.57	826.14	0.1	0.29	826.14	826.89	0.1	0.2
4	1031.67	1035.58	1028.42	0.32	0.7	1030.89	1035.68	0.08	−0.01
5	1311.64	1317.98	1291.29	1.58	2.07	1296.85	1310.97	1.14	0.53
6	555.43	555.43	555.43	0	0	555.43	555.43	0	0
7	909.68	910.58	909.68	0	0.1	909.68	909.68	0	0.1
8	865.95	865.95	865.95	0	0	865.95	866.1	0	−0.02
9	1162.67	1165.3	1162.55	0.01	0.24	1162.76	1168.62	−0.01	−0.28
10	1413.09	1422.81	1395.85	1.24	1.93	1399.15	1407.46	1	1.09
11	1042.11	1042.87	1042.11	0	0.07	1042.11	1042.11	0	0.07
12	819.56	819.56	819.56	0	0	819.56	819.56	0	0
13	1542.86	1545.48	1541.14	0.11	0.28	1541.14	1543.09	0.11	0.15
14	866.37	866.37	866.37	0	0	866.37	866.37	0	0
T.	13,707.84	13,736.35	13,664.36	0.24	0.41	13,675.9	13,711.84	0.17	0.13

Bold values indicate that the best known solution has been found

using probability distributions. At the end of each run, the final solution obtained by the corresponding algorithm is evaluated in the stochastic setting using 100,000 demands realizations. A recourse action as explained in Sect. 3.2 is applied at each evaluation if constraints are violated. The result for each experimental run is the average of these 100,000 evaluations. We considered eight possible demand deviations (5, 10, 15, 20, 25, 30, 35, and 40 %) from the deterministic demands defined in the benchmark. Table 3 shows the results when the demand deviation equals 10 %; the results for the other deviation values (up to 25 %) appear in the Appendix (see Tables 8, 9, 10 and 11):

Regarding the results obtained for the 10 % case, using stochastic information is useful in all instances in the data set except the smallest one (*c50*) for small deviation values, and even in this case, the average gap is less than 1 %. The best solution is also obtained by S-VNS in most cases (18 out of 21). Using S-VNS provides routes which are, on average, 4.39 % shorter than using VNS.

Figure 2 shows the gap between VNS and S-VNS for each considered deviation value. A negative value indicates that S-VNS outperforms VNS, whereas a positive value means that VNS outperforms S-VNS. Incorporating stochastic information into the algorithm leads to better results, up to 25 % deviations. The best gap values using S-VNS are obtained when the deviations are in set {5, 10, 15 %} with an average gap of approximately 4 %. For deviations greater than 30 %, it pays to use the deterministic VNS to obtain shorter routes. This is caused by the frequent acceptance of tentative solutions at the tournament step. Tentative solutions are more likely to be accepted due to the small number of samples used at this step ($s_m = 1$) and the high fluctuations in the cost estimation that happen when the deviations are large. Thus, the algorithm explores many solutions but does not intensify the search.

Solving the stochastic problem with large deviations using S-VNS can be addressed by increasing the sample size at the tournament step, so that new best solutions are likely to

Table 3 Results obtained by VNS and S-VNS on the VRPSD with demands deviation equal to $\pm 10.0\%$

Inst.	VNS		S-VNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
c100	836.15	854.89	834.42	839.30	-0.21	-1.82
c100b	917.80	934.39	867.74	871.54	-5.45	-6.73
c120	1082.50	1158.93	1068.84	1092.90	-1.26	-5.70
c150	1074.79	1112.99	1054.63	1074.84	-1.88	-3.43
c199	1411.19	1446.42	1369.04	1390.10	-2.99	-3.89
c50	540.61	544.57	542.05	547.32	0.27	0.51
c75	885.87	907.64	860.91	875.96	-2.82	-3.49
f134	12,919.32	13,061.02	12,256.46	12,455.29	-5.13	-4.64
f71	256.69	263.11	254.07	254.08	-1.02	-3.43
tai100a	2241.74	2327.00	2142.08	2159.05	-4.45	-7.22
tai100b	2064.23	2142.60	2059.39	2092.91	-0.23	-2.32
tai100c	1467.52	1592.84	1447.99	1465.87	-1.33	-7.97
tai100d	1691.97	1795.02	1645.48	1676.86	-2.75	-6.58
tai150a	3487.99	3627.65	3343.65	3432.99	-4.14	-5.37
tai150b	2928.70	3065.37	2873.89	2933.56	-1.87	-4.30
tai150c	2483.11	2647.05	2487.05	2520.36	0.16	-4.79
tai150d	2911.59	3008.83	2829.63	2891.65	-2.81	-3.89
tai75a	1723.50	1784.70	1668.80	1708.12	-3.17	-4.29
tai75b	1395.13	1456.20	1410.33	1427.31	1.09	-1.98
tai75c	1382.16	1415.87	1359.31	1378.50	-1.65	-2.64
tai75d	1491.79	1526.09	1401.99	1402.17	-6.02	-8.12
Total	45,194.34	46,673.17	43,777.76	44,490.68	-3.13	-4.68
Avg					-2.27	-4.39

Bold entries indicate higher quality solutions

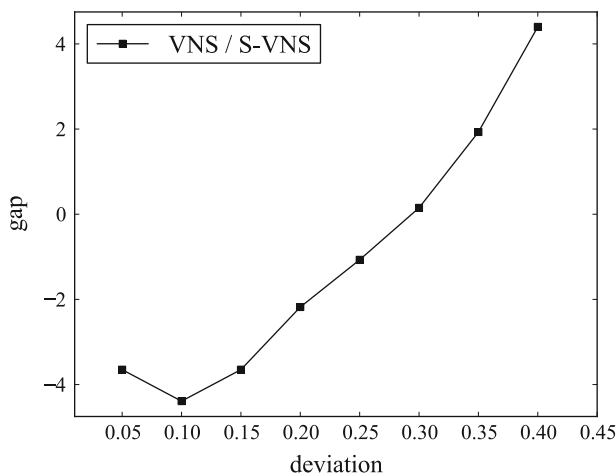


Fig. 2 Average gap between VNS and S-VNS (in %). A negative (*positive*) value denotes that S-VNS obtains shorter (*longer*) routes than VNS

behave “good enough” in a larger and more diverse scenario set. However, sampling requires time, which means that using a larger number of samples in the tournament lead to running for fewer iterations given a fixed run time. If longer run times are not an issue, increasing the sample size can improve the results. Because we aim at solving the dynamic problem too, which involves a limited amount of available time to find a solution, we prefer to keep s_m low.

4.4.2 Standard instances

The current state-of-the-art algorithms for the Christiansen and Lysgaard instances are a simulated annealing (SA) with cyclic-order neighborhoods (Goodson et al. 2012) and a multi-space sampling heuristic (MSSH) (Mendoza and Villegas 2013). The SA executes its outer loop at least 100 iterations, and stops if it is not able to improve the solution in 75 successive iterations. Its inner loop, which follows the standard SA procedure, runs for 7,000 iterations. The MSSH was evaluated by its authors using several maximum number of iterations as stop condition. We compare with the three configurations that provide the higher solution quality (5,000, 10,000, and 20,000 iterations). We set the stop condition of our VNS to 180 CPU seconds, as it seems a good trade-off between solution quality and run time, compared to the previously mentioned algorithms.

Table 4 presents a summary of the results comparing S-VNS with MSSH and SA. The first column contains the same performance measures analyzed in Mendoza and Villegas (2013): (1) the average gap between the best solution found for each instance and the best known solution, (2) the average and the maximum gaps between the average result over 10 runs for each instance and the best known solution, (3) the average, maximum, and minimum run times measured in CPU seconds, and (4) the number of matched best known solutions. The second column presents the results of S-VNS, columns 3–5 show the results obtained by MSSH with three different stop conditions, and column six reports the results obtained by SA. The complete results are included in the Appendix (see Table 12).

S-VNS matched 32 out of 40 best known solutions for this benchmark, matching the results of the most successful algorithm in this context, SA. Regarding the average results, S-VNS obtains results between those of SA and MSSH with 10,000 iterations (0.30 vs. 0.35 and 0.25 %, respectively), and it requires less computation time than SA, but more (in average) than MSSH. MSSH with 20,000 iterations obtains the best average gap (0.18 %), although it

Table 4 Summary of results obtained by S-VNS, Multi-Space Sampling Heuristic (MSSH) (Mendoza and Villegas 2013), and SA with cyclic-order neighborhoods (Goodson et al. 2012) for the 40 instance benchmark of Christiansen and Lysgaard

Performance measure	S-VNS	MSSH (5000)	MSSH (10,000)	MSSH (20,000)	SA
Best gap	0.07	0.18	0.12	0.07	0.04
Avg. gap	0.30	0.36	0.25	0.18	0.35
Max. gap	1.31	1.81	1.34	1.16	1.89
Avg. time	136.05	51.08	93.43	180.78	268.66
Max. time	180.36	179.61	314.09	782.77	603.80
Min. time	0.36	1.90	3.29	5.91	9.00
Matched BKSs ^a	32	17	21	23	32

^a Best known solutions

also requires longer running times than S-VNS. Considering the gap with respect to the best known solution, we obtain the same result as MSSH with 20,000 iterations (0.07 %) with shorter run times, while SA obtains better results (0.04 %) in approximately twice as long run times. In addition, we were able to provide one new best solution for instance P-n55-k10 (new best cost equal to 742.41).

4.5 VRP with dynamic requests

Because customers with $\tau_i < T \cdot 0.5$ are dynamic, all dynamic customers become known in the first half of the planning horizon, while static customers are known before the beginning of the work day and are usually regarded as left over from the previous day. Other algorithms have tackled these instances, including the ant system (Montemanni et al. 2005), genetic algorithm (GA) and tabu search (Hanshar and Ombuki-Berman 2007), VNS and dynamic adapted particle swarm optimization (DAPSO) (Khouadjia et al. 2012), and multi-environmental multi-swarm optimization (MEMSO) (Khouadjia et al. 2011). We compare our results with those obtained by GA and MEMSO, which provide the best available results. We also include DAPSO in the comparison, because the results of MEMSO were obtained using a parallel model with 8 islands running DAPSO. Although another VNS approach is available (Sarasola et al. 2011; Khouadjia et al. 2012), we do not include its results here because they do not improve over the GA for this set of instances.

As already done in previous work on the dynamic VRP, we split the work day in 25 time periods. Each time period has 5 s to return a solution for the current state of the problem, unlike previous work that uses 60 s (Montemanni et al. 2005) or 30 s (Hanshar and Ombuki-Berman 2007) (or work in which the stop condition is based on the number of solution evaluations and set to 500 (Khouadjia et al. 2012) and 5000 (Khouadjia et al. 2011)).

Table 5 presents the results obtained by D-VNS and compares them with GA, DAPSO, and MEMSO. The best and average costs are reported, with bolded entries indicating the best solutions; asterisks highlight the new best solutions. As this table reveals, D-VNS, GA, and MEMSO obtained the best solution in 8, 3, and 10 instances out of 21, respectively. In addition, the best and average gaps from our best and average costs are shown. The total best cost using D-VNS is lower than the total best costs using GA and DAPSO (2.57 and 5.47%, respectively), and 0.26 % worse than that obtained from using MEMSO. Regarding the total average cost, D-VNS obtains solutions that are 2.44 % shorter (better) than those of GA, 7.14 % shorter than those of DAPSO, and 0.30 % shorter than those of MEMSO. Finally, we provide 7 new best solutions.

Note that, as a result of the amount of time allocated to each time period, the four reported algorithms require different execution times. D-VNS runs through the 25 time periods in $25 \times 5 \text{ s} = 125 \text{ s}$, which results in $2625 \text{ s} = 43.75 \text{ min}$ to solve the 21 instances (NB., the complete running time required by the program actually amounts to 44.66 min). A GA execution lasts $25 \times 30 \text{ s} = 750 \text{ s}$, so that 15,750 s (262.50 min) are necessary to solve the 21 instances. The DAPSO results have been previously reported for a single swarm (Khouadjia et al. 2012), using 500 evaluations per time period. The required time for the whole benchmark was 113.07 min. No results related to the run time are available for MEMSO using 8 swarms and 5000 evaluations per time period. Because we are comparing results obtained by different hardware (D-VNS runs on Core2 Quad 2.66 GHz, GA on Pentium 4 2.8 GHz, and DAPSO on Xeon 3 GHz), we also normalize the run time using a benchmark to compare the processors (Geekbench benchmark 2010). Applying a normalization factor, we obtain that GA and DAPSO would require 73.50 and 59.35 min, respectively, if executed on our Core2

Table 5 Best and average route length obtained by VNS, GA (Hanshar and Ombuki-Berman 2007), DAPSO (Khouadjia et al. 2012), and MEMSO (Khouadjia et al. 2011)

Inst.	VNS ^a		GA ^b		DAPSO ^c		MEMSO ^d	
	Best	Avg.	Best	Avg.	Best	Avg.	Best	Avg.
c100	943.87*	985.94	961.10	987.59	988.27	1051.50	949.83	968.92
c100b	883.91	910.67	881.92	900.94	924.32	964.47	864.19	878.81
c120	1219.73	1319.39	1303.59	1390.58	1276.88	1457.22	1164.63	1284.62
c150	1253.66*	1323.40	1348.88	1386.93	1371.08	1470.95	1274.33	1327.24
c199	1538.87*	1608.40	1654.51	1758.51	1640.40	1818.55	1600.57	1649.17
c50	592.60	618.44	570.89	593.42	575.89	632.38	577.60	592.95
c75	940.25	994.89	981.57	1013.45	970.45	1031.76	928.53	962.54
f134	14,559.58*	15,423.87	15,135.51	16,083.56	15,875.00	16,645.89	14,814.10	16,083.82
f71	273.08	291.21	301.79	309.94	279.52	312.35	283.43	294.85
tai100a	2134.95*	2246.88	2232.71	2295.61	2249.84	2370.58	2170.54	2213.75
tai100b	2126.68	2204.05	2147.70	2215.93	2238.42	2385.54	2093.54	2190.01
tai100c	1517.57	1650.93	1541.28	1622.66	1532.56	1627.32	1491.13	1553.55
tai100d	1807.86	1950.62	1834.60	1912.43	1955.06	2123.90	1732.38	1895.42
tai150a	3274.78	3408.89	3328.85	3501.83	3400.33	3612.79	3253.77	3369.48
tai150b	2819.46*	2902.42	2933.40	3115.39	3013.99	3232.11	2865.17	2959.15
tai150c	2491.58*	2643.11	2612.68	2743.55	2714.34	2875.93	2510.13	2644.69
tai150d	2927.21	3006.04	2950.61	3045.16	3025.43	3347.60	2872.80	3006.88
tai75a	1833.18	1913.51	1782.91	1856.66	1816.07	1935.28	1785.11	1837.00
tai75b	1460.65	1496.77	1464.56	1527.77	1447.39	1484.73	1398.68	1425.80
tai75c	1558.05	1616.87	1440.54	1501.91	1481.35	1664.40	1490.32	1532.45
tai75d	1428.74	1452.73	1399.83	1422.27	1414.28	1493.47	1342.26	1448.19
Total	47,586.24	49,969.02	48,809.43	51,186.09	50,190.87	53,538.72	47,463.04	50,119.29
Gap (%)			2.57	2.44	5.47	7.14	−0.26	0.30

^a Run time: 44.66 min^b Run time: 262.50 min (norm. 73.50 min)^c Run time: 113.07 min (norm. 59.35 min)^d Run time: unknown

Quad platform. In addition to obtaining better solution costs, D-VNS is faster than GA and DAPSO, both considering the run time and the normalized run time.

In summary, our approach outperforms the best sequential algorithm in existing literature (GA) and offers results very close (or even better in the average case) to those of MEMSO, a powerful parallel algorithm. In both cases, D-VNS runs faster under a fair comparison with respect to the run time than the other metaheuristic algorithms.

4.6 VRP with stochastic demand and dynamic requests

In this section, we study the impact of dealing with stochastic demands in the dynamic VRP. Following the experimental settings for the VRPDR (see Sect. 4.5), SD-VNS may use 5 s per time period, and the work day is split in 25 time periods. We compare SD-VNS with D-VNS by evaluating the final solutions provided by each algorithm on 100, 000

demand scenarios. Each solution is evaluated 100,000 times using a different demand realization in each case. The result for each experiment is the average of these 100,000 evaluations.

As explained in former sections, dealing with stochastic demands can lead to a violation of the capacity constraint. Therefore, it is necessary to take some recourse to ensure that the service is completed. In this case, because the tour duration constraint is the length of the work day and no restriction applies to the number of vehicles, a new vehicle is released from the depot as soon as another vehicle is unable to fulfill customer demand. This new vehicle serves the remaining unserved customers in the tour. In addition, the impossibility of serving the remaining customers in a route due to the time constraints may be detected if it is already not possible to release a new vehicle that can serve the remaining customers and go back to the depot before the end of the work day. In this case, a constraint is inevitably violated, and the solution is infeasible. Therefore, in addition to the best and average results, columns 4 and 8 (Inf.) in this section's tables report the percentage of infeasible solutions (though we include them in the reported results) and columns 5 and 9 compute the excess route duration needed to serve the remaining customers after the work day has finished.

Following a procedure similar to that we applied in Sect. 4.4, we tested D-VNS and SD-VNS using eight deviation values (5–40 %) for the dynamic instances defined by Kilby (1998). Table 6 shows the results for the 10 % case, and the rest (up to 25 %) are included in the Appendix (see Tables 13, 14, 15, 16). These findings show that SD-VNS provides shorter best and average results than D-VNS and obtains a lower percentage of infeasible solutions and less excess route duration. In instances such as c150, f71, tai100a, tai150c, and tai75d, SD-VNS successfully obtains solutions that do not violate the work day constraint, whereas D-VNS cannot. In other instances such as c120, tai100b, tai150a, and tai150b, SD-VNS reduces the number of infeasible solutions to half the level produced by D-VNS while also reducing the excess route duration. On a few instances, such as tai100d, tai75a, and tai75c, SD-VNS obtains more infeasible solutions than D-VNS. Figure 3 shows the total reduction percentage in the excess route duration obtained by SD-VNS compared with D-VNS for the analyzed deviation values.

Last, we have also tested whether D-VNS and SD-VNS take advantage of longer run times, on the premise that the sampling mechanism requires execution time, and thus slows down the stochastic algorithm. We tested D-VNS and SD-VNS using 5 and 15 s per time period on all instances in the benchmark. D-VNS solves the dynamic problem, whereas SD-VNS solves the stochastic and dynamic problem, using deviations in [5–40 %]. The obtained solutions are evaluated in both cases using 100,000 samples for each demand deviation. Table 7 shows the average results for all instances. If each time period is assigned 5 s run-time, using stochastic information proves useful when the deviation is less than or equal to 25 %, whereas using the optimization algorithm *as is* (regardless of the data stochasticity) is beneficial when the deviation is higher. If 15 s per time period are used, SD-VNS obtains better results than D-VNS for deviations up to 30 %. In addition, a larger gap between D-VNS and SD-VNS can be observed for all deviations. Letting the algorithms run longer proves beneficial for the stochastic algorithm.

In addition, we have evaluated three ways to deal with infeasible solutions, as explained in Sect. 3.2. Our *correct-one-penalize-all* strategy provides an important solution improvement (gap = −31.50 %) compared with the *correct-all* strategy. Penalizing all solutions also improves the *correct-all* strategy to a lesser degree (gap = −30.38 %).

Table 6 Results obtained by D-VNS and SD-VNS on the VRPSDDR with demands deviation equal to $\pm 10.0\%$

Inst.	D-VNS			SD-VNS			Excess time	Inf.	Avg.	Inf.	Excess time	Best gap	Avg. gap
	Best	Avg.	Inf.	Best	Avg.	Inf.							
c100	959.49	1008.57	0.0	971.51	1027.07	0.0	0.0				0.0	1.25	1.83
c100b	911.16	972.96	0.0	893.14	930.87	0.0	0.0				0.0	-1.98	-4.33
c120	1292.29	1423.00	6.7	1249.99	1396.98	3.3	0.0				0.0	-3.27	-1.83
c150	1332.98	1378.98	3.3	1302.65	1394.21	0.0	0.0				0.0	-2.27	1.10
c199	1639.38	1696.93	0.0	1650.32	1723.42	0.0	0.0				0.0	0.67	1.56
c50	610.71	639.73	0.0	604.96	650.22	0.0	0.0				0.0	-0.94	1.64
c75	978.97	1041.45	0.0	991.19	1040.67	0.0	0.0				0.0	1.25	-0.07
f134	16,267.66	17,477.40	0.0	14,926.32	15,843.30	0.0	0.0				0.0	-8.25	-9.35
f71	277.69	300.73	3.3	277.54	297.61	0.0	0.0				0.0	-0.05	-1.04
tai100a	2280.05	2374.17	6.7	2238.52	2340.07	0.0	0.0				0.0	-1.82	-1.44
tai100b	2211.52	2354.31	50.0	2207.39	2329.05	23.3	0.0				0.0	-0.19	-1.07
tai100c	1515.95	1734.86	0.0	1516.20	1588.87	0.0	0.0				0.0	0.02	-8.42
tai100d	1891.05	2021.27	13.3	1911.66	2030.33	23.3	1.5				0.9	1.09	0.45
tai150a	3612.29	3760.28	30.0	3517.01	3676.67	10.0	0.9				0.3	-2.64	-2.22
tai150b	3008.74	3222.06	33.3	2924.99	3074.33	26.7	0.3				0.0	-2.78	-4.59
tai150c	2633.59	2877.50	3.3	2645.63	2839.77	0.0	0.0				0.0	0.46	-1.31
tai150d	3159.33	3345.56	3.3	3122.05	3340.87	10.0	0.0				0.0	-1.18	-0.14
tai75a	1916.82	1991.74	13.3	1846.30	1928.38	20.0	2.2				0.0	-3.68	-3.18
tai75b	1469.81	1533.14	0.0	1475.11	1538.79	0.0	0.0				0.0	0.36	0.37
tai75c	1588.84	1705.69	0.0	1514.22	1659.38	10.0	0.7				0.0	-4.70	-2.72
tai75d	1502.52	1543.63	3.3	1519.28	1689.24	0.0	0.0				0.0	1.12	9.43
Total	51,060.83	54,403.97	10.73	49,305.96	52,340.11		5.74					-3.44	-3.79
Avg			8.1			6.0						-1.31	-1.21

Bold entries indicate higher quality solution

Fig. 3 Reduction of the excess route duration obtained by SD-VNS compared with D-VNS (in %)

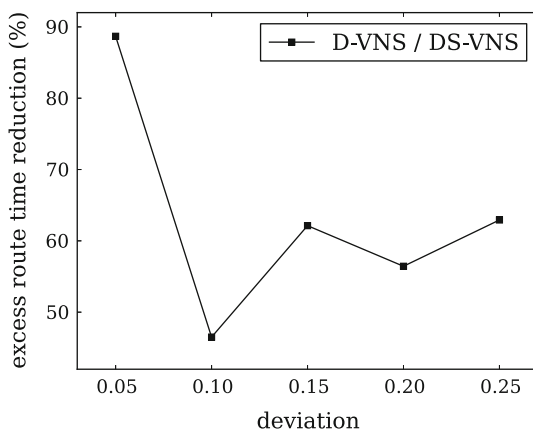


Table 7 Average gap between D-VNS and SD-VNS (in %)

Deviation	Gap SD-VNS to D-VNS	
	5 s	15 s
0.05	−0.74	−1.81
0.10	−1.21	−2.18
0.15	−0.91	−1.59
0.20	−0.53	−1.39
0.25	−0.11	−0.93
0.30	0.47	−0.36
0.35	1.28	0.42
0.40	2.39	1.49

A negative (positive) value denotes that SD-VNS obtains shorter (longer) routes than D-VNS

5 Conclusions

In this work, we have presented four VNS-based approaches for the VRP with stochastic demand and/or dynamic requests.

Regarding the static and stochastic VRP, we present a comparison of VNS and S-VNS, showing how to use stochastic information in the S-VNS to improve the results for relatively small deviation values (deviation in [5, 15] %). In the case of greater deviation values, S-VNS receives too much random information, and using the deterministic VNS yields better results. In addition, we compared our S-VNS with the best heuristics for the VRPSD published in the literature. Our method matched 32 out of 40 best known solutions and also found one new best solution.

In the case of the dynamic and deterministic VRP, we improved on the current state of the art with 7 new best solutions and obtained results that are competitive with MEMSO (a parallel multi-swarm PSO). We also outperform all other algorithms in existing literature (both in time and solution quality).

We have combined the stochastic and dynamic frameworks in one single algorithm, SD-VNS, which can deal with stochastic demands and dynamic requests, and improve the results obtained by the deterministic D-VNS for small deviation values ([5, 15] %). We also have evaluated three ways to deal with infeasible solutions during the optimization process. In this

case, we show that correcting once and using penalties in the SAE is the best policy for this set of instances.

Further work should explore more efficient recourse actions in the SD-VNS, which might reduce the infeasibility of the solutions.

Acknowledgments We acknowledge funds from the Spanish Ministry of Sciences and Innovation European FEDER, under contracts TIN2008-06491-C04-01 (M* project, <http://mstar.lcc.uma.es>) and TIN2011-28194 (roadME project, <http://roadme.lcc.uma.es>), as well as CICE, Junta de Andalucía, under contract P07-TIC-03044 (DIRICOM project, <http://diricom.lcc.uma.es>). Briseida Sarasola received support from grant AP2009-1680 provided by the Spanish government. The financial support by the Austrian Federal Ministry of Economy, Family and Youth and the National Foundation for Research, Technology and Development is gratefully acknowledged.

Appendix

See Tables 8, 9, 10, 11, 12, 13, 14, 15, and 16.

Table 8 Results obtained by VNS and S-VNS on the VRPSD with demands deviation equal to $\pm 5.0\%$

Inst.	VNS		S-VNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
c100	831.63	847.35	830.99	836.37	−0.08	−1.29
c100b	917.15	933.84	866.60	867.61	−5.51	−7.09
c120	1060.93	1126.63	1055.09	1068.11	−0.55	−5.19
c150	1063.28	1095.31	1045.27	1062.10	−1.69	−3.03
c199	1378.46	1419.13	1334.75	1364.66	−3.17	−3.84
c50	533.74	537.19	531.56	541.90	−0.41	0.88
c75	868.14	888.85	854.01	862.17	−1.63	−3.00
f134	12,720.10	12,911.35	11,953.46	12,205.23	−6.03	−5.47
f71	256.03	261.44	254.07	254.09	−0.76	−2.81
tai100a	2164.27	2244.10	2118.07	2137.15	−2.13	−4.77
tai100b	1972.31	2054.72	1975.04	1993.01	0.14	−3.00
tai100c	1430.22	1520.66	1427.92	1437.29	−0.16	−5.48
tai100d	1669.76	1742.96	1633.72	1658.20	−2.16	−4.86
tai150a	3217.31	3434.89	3247.10	3315.65	0.93	−3.47
tai150b	2833.34	2962.42	2843.29	2861.90	0.35	−3.39
tai150c	2439.38	2562.05	2429.51	2458.38	−0.40	−4.05
tai150d	2813.05	2885.85	2708.16	2785.29	−3.73	−3.48
tai75a	1665.74	1731.80	1652.30	1658.72	−0.81	−4.22
tai75b	1365.81	1409.93	1365.00	1366.10	−0.06	−3.11
tai75c	1355.19	1387.61	1345.99	1361.31	−0.68	−1.90
tai75d	1432.55	1458.90	1395.95	1399.33	−2.55	−4.08
Total	43,988.39	45,416.96	42,867.86	43,494.58	−2.55	−4.23
Avg					−1.48	−3.65

Bold entries indicate higher quality solutions

Table 9 Results obtained by VNS and S-VNS on the VRPSD with demands deviation equal to $\pm 15.0\%$

Inst.	VNS		S-VNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
c100	841.91	861.64	836.82	851.20	-0.61	-1.21
c100b	923.11	938.92	882.42	923.42	-4.41	-1.65
c120	1107.91	1187.28	1110.55	1124.36	0.24	-5.30
c150	1084.56	1126.89	1072.79	1098.31	-1.09	-2.54
c199	1435.52	1468.30	1383.10	1413.83	-3.65	-3.71
c50	545.95	549.58	547.33	547.72	0.25	-0.34
c75	898.80	920.76	872.30	889.42	-2.95	-3.40
f134	13,021.14	13,211.82	12,451.19	12,648.42	-4.38	-4.26
f71	257.16	263.59	254.63	258.56	-0.98	-1.91
tai100a	2292.97	2374.95	2165.31	2190.04	-5.57	-7.79
tai100b	2138.32	2207.54	2104.39	2141.01	-1.59	-3.01
tai100c	1513.07	1636.55	1492.98	1511.24	-1.33	-7.66
tai100d	1710.94	1827.84	1663.97	1719.85	-2.75	-5.91
tai150a	3599.70	3755.72	3470.81	3512.24	-3.58	-6.48
tai150b	3015.18	3155.77	2999.90	3100.92	-0.51	-1.74
tai150c	2527.76	2716.18	2517.00	2577.61	-0.43	-5.10
tai150d	2998.04	3106.89	2898.97	2958.64	-3.30	-4.77
tai75a	1774.47	1830.03	1728.85	1761.66	-2.57	-3.74
tai75b	1422.11	1496.26	1450.63	1469.10	2.01	-1.82
tai75c	1400.16	1427.58	1377.53	1392.40	-1.62	-2.46
tai75d	1537.36	1567.79	1484.29	1538.38	-3.45	-1.88
Total	46,046.15	47,631.88	44,765.79	45,628.33	-2.78	-4.21
Avg					-2.01	-3.65

Bold entries indicate higher quality solutions

Table 10 Results obtained by VNS and S-VNS on the VRPSD with demands deviation equal to $\pm 20.0\%$

Inst.	VNS		SVNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
c100	847.50	867.50	851.47	873.05	0.47	0.64
c100b	930.13	945.00	936.56	965.64	0.69	2.18
c120	1133.88	1212.17	1131.85	1171.49	-0.18	-3.36
c150	1096.49	1138.60	1098.44	1118.97	0.18	-1.72
c199	1455.86	1486.09	1417.57	1451.29	-2.63	-2.34
c50	550.47	553.88	548.52	551.77	-0.36	-0.38
c75	910.91	930.69	882.08	908.46	-3.16	-2.39
f134	13,112.33	13,344.61	12,606.26	12,863.04	-3.86	-3.61
f71	258.20	264.12	258.70	270.17	0.19	2.29
tai100a	2332.40	2410.40	2196.17	2273.92	-5.84	-5.66
tai100b	2192.43	2251.10	2129.48	2199.72	-2.87	-2.28
tai100c	1555.68	1667.09	1596.46	1641.00	2.62	-1.56

Table 10 continued

Inst.	VNS		SVNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
tai100d	1733.90	1859.23	1729.85	1767.15	−0.23	−4.95
tai150a	3693.63	3845.86	3525.50	3609.30	−4.55	−6.15
tai150b	3102.69	3232.39	3099.03	3192.81	−0.12	−1.22
tai150c	2583.63	2772.76	2657.90	2745.37	2.87	−0.99
tai150d	3070.49	3181.90	2969.41	3060.63	−3.29	−3.81
tai75a	1816.17	1870.14	1777.38	1801.97	−2.14	−3.65
tai75b	1447.40	1525.52	1457.56	1490.18	0.70	−2.32
tai75c	1413.33	1435.74	1392.87	1416.94	−1.45	−1.31
tai75d	1578.01	1606.13	1513.57	1553.46	−4.08	−3.28
Total	46,815.52	48,400.90	45,776.64	46,926.33	−2.22	−3.05
Avg					−1.29	−2.18

Bold entries indicate higher quality solutions

Table 11 Results obtained by VNS and S-VNS on the VRPSD with demands deviation equal to $\pm 25.0\%$

Inst.	VNS		SVNS		Best gap	Avg. gap
	Best	Avg.	Best	Avg.		
c100	852.40	872.65	865.37	878.69	1.52	0.69
c100b	936.26	950.43	961.19	974.92	2.66	2.58
c120	1158.79	1232.84	1225.95	1246.08	5.80	1.07
c150	1108.02	1148.89	1108.04	1142.13	0.00	−0.59
c199	1472.45	1501.10	1448.28	1486.96	−1.64	−0.94
c50	553.99	557.71	551.29	555.44	−0.49	−0.41
c75	921.61	938.85	895.51	918.33	−2.83	−2.19
f134	13,199.77	13,461.72	12,853.35	13,110.31	−2.62	−2.61
f71	259.25	264.79	267.72	274.55	3.27	3.69
tai100a	2363.37	2439.59	2286.93	2352.56	−3.23	−3.57
tai100b	2231.86	2284.67	2198.30	2290.19	−1.50	0.24
tai100c	1595.47	1690.95	1618.47	1633.26	1.44	−3.41
tai100d	1746.71	1880.13	1748.05	1787.61	0.08	−4.92
tai150a	3780.07	3914.30	3635.42	3717.20	−3.83	−5.04
tai150b	3169.75	3296.09	3186.79	3300.31	0.54	0.13
tai150c	2643.71	2819.49	2729.05	2824.29	3.23	0.17
tai150d	3127.35	3239.01	3085.10	3177.99	−1.35	−1.88
tai75a	1851.44	1902.01	1807.47	1863.77	−2.37	−2.01
tai75b	1473.22	1549.94	1474.91	1553.60	0.11	0.24
tai75c	1422.30	1442.60	1411.06	1429.47	−0.79	−0.91
tai75d	1605.96	1636.68	1569.65	1590.16	−2.26	−2.84
Total	47,473.77	49,024.44	46,927.88	48,107.82	−1.15	−1.87
Avg					−0.20	−1.07

Bold entries indicate higher quality solutions

Table 12 Results obtained by S-VNS, MSSH (Mendoza and Villegas 2013), and SA (Goodson et al. 2012) for the Christiansen and Lysgaard benchmark

Instance	BKS ^a	S-VNS			MSSH (5,000)			MSSH (10,000)			MSSH (20,000)			SA		
		Best	Avg.	Best	Best gap	Avg. gap	Avg.	Best gap	Avg. gap	Avg.	Best gap	Avg. gap	Avg.	Best gap	Avg. gap	Avg. gap
A-n32-k5	853.60	853.60		0.00	0.00	0.21	0.00	0.00	0.14	0.00	0.00	0.06	0.00	0.00	0.00	0.00
A-n33-k5	704.20	704.20	709.40	0.00	0.74	0.91	0.74	0.74	0.74	0.00	0.00	0.54	0.00	0.00	0.24	0.24
A-n33-k6	793.90	793.90	793.90	0.00	0.00	0.07	0.00	0.00	0.02	0.00	0.00	0.02	0.00	0.00	0.01	0.01
A-n34-k5	826.87	826.87	826.87	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.05
A-n36-k5	858.71	858.71	860.22	0.00	0.18	1.62	0.70	0.00	1.34	0.00	0.00	0.80	0.00	0.00	0.09	0.09
A-n37-k5	708.34	708.34	708.34	0.00	0.00	0.16	0.16	0.17	0.17	0.16	0.16	0.16	0.00	0.00	0.19	0.19
A-n37-k6	1030.73	1030.73	1030.73	0.00	0.00	0.03	0.00	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.10	0.10
A-n38-k5	775.13	775.13	775.13	0.00	0.00	0.31	0.17	0.18	0.25	0.00	0.00	0.16	0.00	0.00	0.02	0.02
A-n39-k5	869.18	869.18	869.18	0.00	0.00	1.05	0.03	0.03	0.65	0.03	0.03	0.37	0.00	0.00	0.05	0.05
A-n39-k6	876.60	887.91	888.09	1.29	1.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.78	0.78
A-n44-k6	1025.48	1025.48	1025.98	0.00	0.05	0.21	0.05	0.01	0.09	0.01	0.01	0.05	0.00	0.00	0.41	0.41
A-n45-k6	1026.73	1026.73	1027.36	0.00	0.06	0.05	0.01	0.01	0.01	0.01	0.01	0.01	0.00	0.00	0.12	0.12
A-n45-k7	1264.83	1264.83	1265.47	0.00	0.05	0.37	0.37	0.14	0.48	0.14	0.14	0.30	0.01	0.01	1.89	1.89
A-n46-k7	1002.22	1002.22	1003.36	0.00	0.11	0.00	0.00	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.10	0.10
A-n48-k7	1187.14	1187.14	1187.14	0.00	0.00	1.43	1.28	1.16	1.34	0.92	0.92	1.16	0.00	0.00	0.08	0.08
A-n53-k7	1124.27	1126.58	1129.91	0.21	0.50	0.39	0.39	0.29	0.49	0.13	0.13	0.34	0.00	0.00	0.45	0.45
A-n54-k7	1287.07	1287.66	1287.66	0.05	0.05	0.55	0.35	0.24	0.46	0.06	0.06	0.34	0.00	0.00	0.41	0.41
A-n55-k9	1179.11	1179.11	1188.02	0.00	0.76	0.45	0.00	0.00	0.29	0.00	0.00	0.15	0.00	0.00	0.48	0.48
A-n60-k9	1529.82	1529.82	1538.22	0.00	0.55	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.36	0.36
E-n22-k4	411.57	411.57	411.57	0.00	0.00	0.09	0.00	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00
E-n33-k4	850.27	850.27	850.27	0.00	0.00	0.73	0.53	0.53	0.55	0.33	0.33	0.51	0.00	0.00	0.11	0.11
E-n51-k5	552.26	552.26	553.48	0.00	0.22	0.13	0.00	0.00	0.07	0.00	0.00	0.03	0.00	0.00	0.10	0.10
P-n16-k8	512.82	512.82	512.82	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
P-n19-k2	224.06	224.06	224.06	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Table 12 continued

Instance	BKS ^a	S-VNS		MSSH (5,000)				MSSH (10,000)				MSSH (20,000)				SA	
		Best	Avg.	Best	Avg.	Best gap	Avg. gap	Best gap	Avg. gap	Best gap	Avg. gap	Best gap	Avg. gap	Best gap	Avg. gap	Best gap	Avg. gap
P-n20-k2	233.05	233.05	233.05	0.00	0.00	0.00	0.94	0.00	0.56	0.00	0.23	0.00	0.00	0.00	0.00	0.00	0.00
P-n21-k2	218.96	218.96	218.96	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
P-n22-k2	231.26	231.26	231.26	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
P-n22-k8	681.06	681.06	681.06	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
P-n23-k8	619.52	619.53	620.38	0.00	0.14	0.00	0.01	0.00	0.01	0.00	0.01	0.00	0.00	0.01	0.00	0.00	0.01
P-n40-k5	472.50	472.50	472.50	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
P-n45-k5	533.52	533.52	533.52	0.00	0.00	0.19	0.21	0.19	0.19	0.19	0.19	0.19	0.19	0.19	0.19	0.00	0.68
P-n50-k10	758.76	763.24	767.08	0.59	1.10	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.29	0.71
P-n50-k7	582.37	582.37	584.08	0.00	0.29	1.39	1.81	0.56	0.83	0.46	0.69	0.00	0.00	0.00	0.00	0.00	0.44
P-n50-k8	669.23	670.97	676.61	0.26	1.10	0.00	0.05	0.00	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.09	0.73
P-n51-k10	809.70	809.70	816.00	0.00	0.78	0.05	0.11	0.00	0.04	0.00	0.04	0.00	0.00	0.38	0.90	0.00	0.90
P-n55-k10	742.41	742.41 [*]	747.16	0.00	0.40	0.24	0.74	0.24	0.58	0.08	0.38	0.00	0.00	0.44	1.34	0.00	1.34
P-n55-k15	1068.05	1068.05	1074.05	0.00	0.56	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.39	0.00	0.39
P-n55-k7	588.56	588.56	593.27	0.00	0.80	0.32	0.74	0.18	0.46	0.18	0.38	0.00	0.00	0.00	0.77	0.00	0.77
P-n60-k10	803.60	804.24	810.71	0.08	0.88	0.12	0.18	0.00	0.10	0.00	0.10	0.00	0.00	0.08	0.82	0.00	0.82
P-n60-k15	1085.49	1089.53	1098.62	0.37	1.21	0.00	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.18	1.16	0.00	1.16
Total	31,572.96	31,598.07	31,679.09														
Avg				0.07	0.30	0.18	0.36	0.12	0.25	0.07	0.18	0.07	0.18	0.04	0.35	0.04	0.35
Max				1.29	1.31	1.39	1.81	1.16	1.34	0.92	1.16	0.92	1.16	0.44	1.89	0.44	1.89
Min				0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Bold values mean that the best known solution has been found

^{*} New best solution^a Best known solution

Table 13 Results obtained by D-VNS and SD-VNS on the VRPSDDR with demands deviation equal to $\pm 5.0\%$

Inst.	VNS			SVNS			Excess time	Inf.	Avg.	Excess time	Best gap	Avg. gap
	Best	Avg.	Inf.	Best	Avg.	Inf.						
c100	948.50	1000.71	0.0	972.40	1020.23	0.0	0.0	0.0	2.52	1.95		
c100b	910.41	972.20	0.0	890.00	923.72	0.0	0.0	0.0	-2.24	-4.99		
c120	1270.61	1397.35	3.3	1262.62	1387.53	0.0	0.0	0.0	-0.63	-0.70		
c150	1323.46	1364.52	0.0	1323.99	1384.50	0.0	0.0	0.0	0.04	1.46		
c199	1608.22	1671.00	0.0	1607.42	1691.66	0.0	0.0	0.0	-0.05	1.24		
c50	599.73	632.98	0.0	613.20	643.62	0.0	0.0	0.0	2.25	1.68		
c75	962.36	1025.08	0.0	984.41	1029.66	0.0	0.0	0.0	2.29	0.45		
f134	16,144.22	17,278.62	0.0	14,915.02	15,844.95	0.0	0.0	0.0	-7.61	-8.30		
f71	276.15	298.42	3.3	277.43	298.64	0.0	0.0	0.0	0.46	0.07		
tai100a	2240.78	2309.09	3.3	2178.61	2313.86	0.0	0.0	0.0	-2.77	0.21		
tai100b	2169.34	2288.92	30.0	2185.36	2296.26	16.7	0.1	0.0	0.74	0.32		
tai100c	1504.26	1694.38	0.0	1501.21	1547.73	0.0	0.0	0.0	-0.20	-8.65		
tai100d	1874.84	1995.09	10.0	1851.44	2015.74	16.7	0.1	0.0	-1.25	1.04		
tai150a	3444.04	3613.56	23.3	3430.41	3593.07	30.0	0.2	0.0	-0.40	-0.57		
tai150b	2960.57	3117.96	23.3	2882.64	3053.47	6.7	0.0	0.0	-2.63	-2.07		
tai150c	2589.13	2797.30	3.3	2568.04	2766.29	3.3	0.0	0.0	-0.81	-1.11		
tai150d	3054.74	3212.93	3.3	3059.64	3179.16	6.7	0.0	0.0	0.16	-1.05		
tai75a	1873.55	1946.12	0.0	1802.42	1921.81	0.0	0.0	0.0	-3.80	-1.25		
tai75b	1452.55	1504.42	0.0	1451.64	1496.58	3.3	0.0	0.0	-0.06	-0.52		
tai75c	1575.32	1682.30	0.0	1517.58	1656.33	3.3	0.2	0.0	-3.67	-1.54		
tai75d	1459.06	1493.66	0.0	1463.78	1595.37	0.0	0.0	0.0	0.32	6.81		
Total	50,241.85	53,296.60	5.39	48,739.24	51,660.19		0.61		-2.99	-3.07		
Avg			4.9			4.1			-0.83	-0.74		

Bold entries indicate higher quality solutions

Table 14 Results obtained by D-VNS and SD-VNS on the VRPSDDR with demands deviation equal to $\pm 15.0\%$

Inst.	VNS				SVNS				Excess time	Best gap	Avg. gap
	Best	Avg.	Inf.	Excess time	Best	Avg.	Inf.				
c100	967.62	1015.49	0.0	0.0	978.29	1028.60	6.7	0.0	1.10	1.29	
c100b	918.81	980.37	0.0	0.0	923.19	987.65	0.0	0.0	0.48	0.74	
c120	1304.84	1439.73	6.7	0.3	1242.08	1425.18	3.3	0.1	-4.81	-1.01	
c150	1338.96	1391.00	3.3	0.0	1320.38	1416.22	0.0	0.0	-1.39	1.81	
c199	1664.44	1716.59	3.3	0.0	1639.76	1729.91	0.0	0.0	-1.48	0.78	
c50	619.22	645.34	0.0	0.0	626.41	652.35	0.0	0.0	1.16	1.09	
c75	994.35	1054.35	0.0	0.0	1003.13	1050.88	0.0	0.0	0.88	-0.33	
f134	16,353.37	17,601.69	0.0	0.0	15,115.68	15,696.53	0.0	0.0	-7.57	-10.82	
f71	279.75	302.62	3.3	0.0	287.54	307.43	0.0	0.0	2.79	1.59	
tai100a	2311.37	2421.39	13.3	0.4	2324.97	2455.39	0.0	0.0	0.59	1.40	
tai100b	2264.46	2415.18	60.0	2.8	2254.77	2373.35	43.3	0.8	-0.43	-1.73	
tai100c	1530.06	1771.86	0.0	0.0	1572.36	1640.48	0.0	0.0	2.76	-7.41	
tai100d	1903.98	2046.96	20.0	0.8	1870.22	2054.18	26.7	1.7	-1.77	0.35	
tai150a	3731.42	3878.55	30.0	4.6	3567.61	3770.05	10.0	1.7	-4.39	-2.80	
tai150b	3047.30	3299.34	33.3	6.6	3020.19	3166.55	40.0	0.8	-0.89	-4.02	
tai150c	2677.55	2945.02	20.0	0.1	2676.14	2886.33	16.7	0.1	-0.05	-1.99	
tai150d	3222.18	3432.60	13.3	0.6	3150.36	3337.36	10.0	0.4	-2.23	-2.77	
tai75a	1965.24	2035.54	66.7	0.2	1915.50	2008.58	46.7	0.6	-2.53	-1.32	
tai75b	1511.62	1570.35	0.0	0.0	1491.95	1582.27	0.0	0.0	-1.30	0.76	
tai75c	1610.32	1720.28	3.3	0.0	1498.54	1657.15	3.3	0.1	-6.94	-3.67	
tai75d	1537.04	1582.04	3.3	0.0	1592.09	1723.59	3.3	0.0	3.58	8.95	
Total	51,753.89	55,266.28		16.45	50,071.17	52,950.04		6.23	-3.25	-4.19	
Avg			13.3				10.0		-1.07	-0.91	
Bold entries indicate higher quality solutions											

Bold entries indicate higher quality solutions

Table 15 Results obtained by D-VNS and SD-VNS on the VRPSDDR with demands deviation equal to $\pm 20.0\%$

Inst.	VNS			SVNS			Excess time	Inf.	Avg.	Inf.	Excess time	Best gap	Avg. gap
	Best	Avg.	Inf.	Excess time	Best	Avg.							
c100	971.02	1021.78	0.0	0.0	981.00	1033.56	0.0	0.0	1033.56	0.0	0.0	1.03	1.15
c100b	931.44	991.87	0.0	0.0	975.20	1036.89	0.0	0.0	1036.89	0.0	0.0	4.70	4.54
c120	1314.37	1454.96	6.7	0.4	1267.56	1465.43	6.7	0.0	1465.43	6.7	0.0	-3.56	0.72
c150	1346.65	1401.99	3.3	0.0	1355.61	1437.23	0.0	0.0	1437.23	0.0	0.0	0.67	2.51
c199	1684.11	1732.48	3.3	0.0	1634.61	1748.37	0.0	0.0	1748.37	0.0	0.0	-2.94	0.92
c50	624.99	650.00	0.0	0.0	624.33	653.11	3.3	0.0	653.11	3.3	0.0	-0.11	0.48
c75	1007.36	1065.85	3.3	0.0	994.62	1059.30	0.0	0.0	1059.30	0.0	0.0	-1.26	-0.61
f134	16,450.35	17,685.78	0.0	0.0	14,996.55	15,791.86	0.0	0.0	15,791.86	0.0	0.0	-8.84	-10.71
f71	281.83	303.87	3.3	0.0	290.12	308.63	0.0	0.0	308.63	0.0	0.0	2.94	1.57
tai100a	2334.32	2458.14	13.3	0.6	2330.42	2471.48	13.3	0.4	2471.48	13.3	0.4	-0.17	0.54
tai100b	2310.38	2461.42	66.7	3.9	2280.49	2410.84	40.0	0.1	2410.84	40.0	0.1	-1.29	-2.06
tai100c	1549.62	1800.16	0.0	0.0	1651.45	1741.14	0.0	0.0	1741.14	0.0	0.0	6.57	-3.28
tai100d	1930.93	2071.82	20.0	1.5	1973.68	2143.52	16.7	1.0	2143.52	16.7	1.0	2.21	3.46
tai150a	3806.02	3968.61	30.0	5.9	3672.11	3867.41	13.3	3.9	3867.41	13.3	3.9	-3.52	-2.55
tai150b	3089.09	3359.67	40.0	7.7	3044.39	3179.58	30.0	0.3	3179.58	30.0	0.3	-1.45	-5.36
tai150c	2726.01	3001.20	20.0	0.7	2761.06	2996.97	16.7	1.1	2996.97	16.7	1.1	1.29	-0.14
tai150d	3273.99	3492.58	16.7	0.8	3138.82	3432.17	16.7	0.3	3432.17	16.7	0.3	-4.13	-1.73
tai75a	1995.60	2075.10	66.7	0.7	1941.39	2044.70	40.0	2.1	2044.70	40.0	2.1	-2.72	-1.47
tai75b	1543.60	1596.18	6.7	0.0	1521.61	1610.96	3.3	0.2	1610.96	3.3	0.2	-1.42	0.93
tai75c	1631.51	1733.30	3.3	0.0	1551.52	1651.97	6.7	0.5	1651.97	6.7	0.5	-4.90	-4.69
tai75d	1577.36	1619.77	3.3	0.1	1597.43	1695.52	0.0	0.0	1695.52	0.0	0.0	1.27	4.68
Total	52,380.53	55,946.53		22.45	50,583.97	53,780.62		9.78				-3.43	-3.87
Avg			14.6				9.8					-0.74	-0.53

Bold entries indicate higher quality solutions

Table 16 Results obtained by D-VNS and SD-VNS on the VRPSDDR with demands deviation equal to $\pm 25.0\%$

Inst.	VNS			SVNS			Excess time	Inf.	Avg.	Best	Excess time	Inf.	Avg.	Best gap	Avg. gap
	Best	Avg.	Inf.	Excess time	Best	Avg.									
c100	974.37	1027.60	6.7	0.0	987.92	1045.32	0.0	0.0	1045.32	987.92	0.0	0.0	1045.32	1.39	1.72
c100b	945.88	1003.82	0.0	0.0	1009.93	1056.08	0.0	0.0	1056.08	1009.93	0.0	0.0	1056.08	6.77	5.21
c120	1325.49	1468.80	10.0	0.4	1292.84	1463.22	10.0	0.7	1463.22	1292.84	0.7	0.7	1463.22	-2.46	-0.38
c150	1355.60	1412.13	6.7	0.0	1347.55	1439.78	3.3	0.0	1439.78	1347.55	0.0	0.0	1439.78	-0.59	1.96
c199	1700.43	1746.31	3.3	0.0	1706.46	1792.90	0.0	0.0	1792.90	1706.46	0.0	0.0	1792.90	0.35	2.67
c50	629.29	654.06	0.0	0.0	616.81	659.61	0.0	0.0	659.61	616.81	0.0	0.0	659.61	-1.98	0.85
c75	1018.31	1075.82	3.3	0.0	1031.28	1082.84	0.0	0.0	1082.84	1031.28	0.0	0.0	1082.84	1.27	0.65
f134	16,545.80	17,760.76	0.0	0.0	14,935.16	16,045.39	0.0	0.0	16,045.39	14,935.16	0.0	0.0	16,045.39	-9.73	-9.66
f71	283.25	304.71	3.3	0.0	288.61	308.90	6.7	0.0	308.90	288.61	0.0	0.0	308.90	1.89	1.38
tai100a	2367.04	2491.43	13.3	0.9	2316.75	2541.08	13.3	0.2	2541.08	2316.75	0.2	0.2	2541.08	-2.12	1.99
tai100b	2350.37	2498.65	83.3	4.9	2319.22	2436.50	26.7	0.4	2436.50	2319.22	0.4	0.4	2436.50	-1.33	-2.49
tai100c	1573.01	1822.54	3.3	0.0	1678.70	1737.31	3.3	0.0	1737.31	1678.70	0.0	0.0	1737.31	6.72	-4.68
tai100d	1942.20	2087.65	23.3	1.9	1980.45	2172.46	16.7	1.9	2172.46	1980.45	1.9	1.9	2172.46	1.97	4.06
tai150a	3892.83	4039.32	33.3	6.8	3799.46	3952.34	20.0	1.9	3952.34	3799.46	1.9	1.9	3952.34	-2.40	-2.15
tai150b	3138.42	3409.01	40.0	8.7	3147.28	3281.72	40.0	0.9	3281.72	3147.28	0.9	0.9	3281.72	0.28	-3.73
tai150c	2775.85	3048.38	20.0	1.6	2851.22	3075.45	20.0	1.4	3075.45	2851.22	1.4	1.4	3075.45	2.72	0.89
tai150d	3322.58	3538.61	20.0	1.1	3363.83	3506.24	13.3	0.0	3506.24	3363.83	0.0	0.0	3506.24	1.24	-0.91
tai75a	2032.38	2111.34	66.7	1.4	1952.52	2062.68	46.7	1.5	2062.68	1952.52	1.5	1.5	2062.68	-3.93	-2.30
tai75b	1557.69	1616.51	10.0	0.0	1538.36	1622.53	0.0	0.0	1622.53	1538.36	0.0	0.0	1622.53	-1.24	0.37
tai75c	1649.07	1744.27	3.3	0.0	1555.80	1669.90	26.7	1.3	1669.90	1555.80	1.3	1.3	1669.90	-5.66	-4.26
tai75d	1611.35	1649.77	3.3	0.2	1627.10	1756.67	10.0	0.0	1756.67	1627.10	0.0	0.0	1756.67	0.98	6.48
Total	52,991.21	56,511.49		28.05	51,347.26	54,708.91		10.40		51,347.26		12.2		-3.10	-3.19
Avg			16.8											-0.28	-0.11

Bold entries indicate higher quality solutions

References

- Ak, A., & Erera, A. L. (2007). A paired-vehicle recourse strategy for the vehicle-routing problem with stochastic demands. *Transportation Science*, 41(2), 222–237.
- Augerat, P., Belenguer, J. M., Benavent, E., Corberán, Á., & Naddef, D. (1998). Separating capacity constraints in the CVRP using tabu search. *European Journal of Operational Research*, 106(2–3), 546–557.
- Bent, R., & Van Hentenryck, P. (2004). Scenario-based planning for partially dynamic vehicle routing with stochastic customers. *Operations Research*, 52(6), 977–987.
- Bianchi, L., Birattari, M., Chiarandini, M., Manfrin, M., Mastrolilli, M., Paquete, L., et al. (2006). Hybrid metaheuristics for the vehicle routing problem with stochastic demands. *Journal of Mathematical Modelling and Algorithms*, 5(1), 91–110.
- Branchini, R. M., Armentano, V. A., & Løkketangen, A. (2009). Adaptive granular local search heuristic for a dynamic vehicle routing problem. *Computers & Operations Research*, 36(11), 2955–2968.
- Bräysy, O. (2003). A reactive variable neighborhood search for the vehicle routing problem with time windows. *INFORMS Journal on Computing*, 15(4), 347–368.
- Christiansen, C. H., & Lysgaard, J. (2007). A branch-and-price algorithm for the capacitated vehicle routing problem with stochastic demands. *Operations Research Letters*, 35(6), 773–781.
- Christofides, N., & Eilon, S. (1969). An algorithm for the vehicle-dispatching problem. *OR*, 20(3), 309–318.
- Christofides, N., Mingozzi, A., & Toth, P. (1979). *The vehicle routing problem, chap. 11*. London: Wiley.
- Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568–581.
- Croes, A. (1958). A method for solving traveling salesman problems. *Operations Research*, 5, 791–812.
- Dantzig, G. B., & Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1), 80–91.
- Erera, A. L., Morales, J. C., & Savelsbergh, M. W. P. (2010). The vehicle routing problem with stochastic demand and duration constraints. *Transportation Science*, 44(4), 474–492.
- Fisher, M. L., & Jaikumar, R. (1981). A generalized assignment heuristic for vehicle routing. *Networks*, 11(2), 109–124.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. San Francisco: W. H. Freeman.
- Geekbench benchmark. (2010). <http://www.primatelabs.ca/>.
- Gendreau, M., Guertin, F., Potvin, J. Y., & Taillard, E. (1999). Parallel tabu search for real-time vehicle routing and dispatching. *Transportation Science*, 33(4), 381–390.
- Gendreau, M., Laporte, G., & Séguin, R. (1996). A tabu search heuristic for the vehicle routing problem with stochastic demands and customers. *Operations Research*, 44(3), 469–477.
- Goodson, J. C., Ohlmann, J. W., & Thomas, B. W. (2012). Cyclic-order neighborhoods with application to the vehicle routing problem with stochastic demand. *European Journal of Operational Research*, 217(2), 312–323.
- Goodson, J. C., Ohlmann, J. W., & Thomas, B. W. (2013). Rollout policies for dynamic solutions to the multivehicle routing problem with stochastic demand and duration limits. *Operations Research*, 61(1), 138–154.
- Gutjahr, W. J., Katzensteiner, S., & Reiter, P. (2007). A VNS algorithm for noisy problems and its application to project portfolio analysis. In *SAGA 2007, lecture notes in computer science, vol. 4665*, (pp. 93–104). Berlin: Springer.
- Hansen, P., & Mladenović, N. (1999). An introduction to variable neighborhood search. In S. Voß, S. Martello, I. Osman, & C. Roucairol (Eds.), *Metaheuristics: Advances and trends in local search paradigms for optimization, chap. 30* (pp. 433–458). Dordrecht: Kluwer.
- Hanshar, F. T., & Ombuki-Berman, B. M. (2007). Dynamic vehicle routing using genetic algorithms. *Applied Intelligence*, 27, 89–99.
- Hvattum, L. M., Løkketangen, A., & Laporte, G. (2006). Solving a dynamic and stochastic vehicle routing problem with a sample scenario hedging heuristic. *Transportation Science*, 40(4), 421–438.
- Khouadjia, M. R., Sarasola, B., Alba, E., Jourdan, L., & Talbi, E. G. (2011). Multi-environmental cooperative parallel metaheuristics for solving dynamic optimization problems. In *IPDPS Workshops*, (pp. 395–403). IEEE.
- Khouadjia, M. R., Sarasola, B., Alba, E., Jourdan, L., & Talbi, E. G. (2012). A comparative study between dynamic adapted PSO and VNS for the vehicle routing problem with dynamic requests. *Applied Soft Computing*, 12(4), 1426–1439.
- Kilby, P., Prosser, P., & Shaw, P. (1998). *Dynamic VRPs: A study of scenarios*. Tech. rep.: University of Strathclyde, UK.
- Kritzing, S., Tricoire, F., Doerner, K. F., Hartl, R. F., & Stützel, T. (2012). *A unified framework for routing problems with fixed fleet size*. Technical Report, submitted.

- Kuo, Y., & Wang, C. C. (2012). A variable neighborhood search for the multi-depot vehicle routing problem with loading cost. *Expert Systems with Applications*, 39(8), 6949–6954.
- Kytöjoki, J., Nuortio, T., Bräysy, O., & Gendreau, M. (2007). An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. *Computers & Operations Research*, 34(9), 2743–2757.
- Laporte, G., Louveaux, F. V., & van Hamme, L. (2002). An integer l-shaped algorithm for the capacitated vehicle routing problem with stochastic demands. *Operations Research*, 50(3), 415–423.
- Lecluyse, C., Van Woensel, T., & Peremans, H. (2009). Vehicle routing with stochastic time-dependent travel times. *4OR*, 7(4), 363–377.
- Lei, H., Laporte, G., & Guo, B. (2011). The capacitated vehicle routing problem with stochastic demands and time windows. *Computers & Operations Research*, 38(12), 1775–1783.
- Li, X., Tian, P., & Leung, S. C. (2010). Vehicle routing problems with time windows and stochastic travel and service times: Models and algorithm. *International Journal of Production Economics*, 125(1), 137–145.
- Lorini, S., Potvin, J. Y., & Zufferey, N. (2011). Online vehicle routing and scheduling with dynamic travel times. *Computers & Operations Research*, 38(7), 1086–1090.
- Mendoza, J. E., Rousseau, L. M., & Villegas, J. G. A hybrid metaheuristic for the vehicle routing problem with stochastic demand and duration constraints. Under review.
- Mendoza, J. E., Castanier, B., Guéret, C., Medaglia, A. L., & Velasco, N. (2010). A memetic algorithm for the multi-compartment vehicle routing problem with stochastic demands. *Computers & Operations Research*, 37(11), 1886–1898.
- Mendoza, J. E., & Villegas, J. G. (2013). A multi-space sampling heuristic for the vehicle routing problem with stochastic demands. *Optimization Letters*, 7(7), 1503–1516.
- Montemanni, R., Gambardella, L. M., Rizzoli, A. E., & Donati, A. V. (2005). Ant colony system for a dynamic vehicle routing problem. *Journal of Combinatorial Optimization*, 10(4), 327–343.
- Mu, Q., Fu, Z., Lysgaard, J., & Eglese, R. (2011). Disruption management of the vehicle routing problem with vehicle breakdown. *Journal of the Operational Research Society*, 62(4), 742–749.
- Pillac, V., Gendreau, M., Guéret, C., & Medaglia, A. L. (2013). A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 225(1), 1–11.
- Polacek, M., Hartl, R. F., Doerner, K., & Reimann, M. (2004). A variable neighborhood search for the multi depot vehicle routing problem with time windows. *Journal of Heuristics*, 10(6), 613–627.
- Potvin, J. Y., & Rousseau, J. M. (1995). An exchange heuristic for routing problems with time windows. *Journal of the Operational Research Society*, 46, 1433–1446.
- Potvin, J. Y., Xu, Y., & Benyahia, I. (2006). Vehicle routing and scheduling with dynamic travel times. *Computers & Operations Research*, 33, 1129–1137.
- Psaraftis, H. N. (1988). *Dynamic vehicle routing problems (vol. 16)*. North Holland.
- Psaraftis, H. N. (1995). Dynamic vehicle routing: Status and prospects. *Annals of Operations Research*, 61, 143–164.
- Sarasola, B., Khoudjia, M. R., Alba, E., Jourdan, L., & Talbi, E. G. (2011). Flexible variable neighborhood search in dynamic vehicle routing. In *InvoApplications (1), lecture notes in computer science, vol. 6624*, (pp. 344–353). Berlin: Springer.
- Schilde, M., Doerner, K. F., & Hartl, R. F. (2011). Metaheuristics for the dynamic stochastic dial-a-ride problem with expected return transports. *Computers & Operations Research*, 38(12), 1719–1730.
- Secomandi, N. (2001). A rollout policy for the vehicle routing problem with stochastic demands. *Operations Research*, 49(5), 796–802.
- Secomandi, N., & Margot, F. (2009). Reoptimization approaches for the vehicle-routing problem with stochastic demands. *Operations Research*, 57(1), 214–230.
- Taillard, É. D. (1993). Parallel iterative search methods for vehicle routing problems. *Networks*, 23(8), 661–673.
- Taillard, É. D., Badeau, P., Gendreau, M., Guertin, F., & Potvin, J. Y. (1997). A tabu search heuristic for the vehicle routing problem with soft time windows. *Transportation Science*, 31(2), 170–186.
- Ulmer, M. W., Brinkmann, J., & Mattfeld, D. C. (2013). Anticipatory planning for courier, express and parcel services. To appear in Proceedings of Logistik Management.
- Zachariadis, E. E., & Kiranoudis, C. T. (2010). An open vehicle routing problem metaheuristic for examining wide solution neighborhoods. *Computers & Operations Research*, 37(4), 712–723.
- Zhang, T., Chaovalitwongse, W., & Zhang, Y. (2012). Scatter search for the stochastic travel-time vehicle routing problem with simultaneous pick-ups and deliveries. *Computers & Operations Research*, 39(10), 2277–2290.